

UNIVERSITÀ DEGLI STUDI DI MILANO
FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA
“GIOVANNI DEGLI ANTONI”



LAUREA MAGISTRALE IN
SICUREZZA INFORMATICA

TODO: TITOLO DELLA TESI

Autore: Enea Manzi
Matricola: 65935A

Relatore: Prof. Marco Anisetti
Correlatore: Prof. Claudio Agostino Ardagna

A.A. 2025-2026

Abstract

Ringraziamenti

Indice

Abstract	ii
Ringraziamenti	iii
Elenco delle figure	vi
Elenco delle tabelle	vii
Elenco dei codici	viii
1 Introduzione	1
1.1 Motivazione e Contesto	1
1.2 Definizione del Problema e Gap nella Ricerca	2
1.3 Obiettivi della Ricerca	3
1.4 Organizzazione della Tesi	4
2 Background e Stato dell'Arte	5
2.1 Paradigmi delle Architetture API Moderne	5
2.1.1 Evoluzione dei Sistemi Distribuiti	5
2.1.2 Tassonomia degli Stili Architettureali e di Comunicazione	6
2.1.3 Il Fenomeno dell'API Sprawl	7
2.2 Il Pattern dell'API Gateway in Ottica Zero-Trust	8
2.2.1 Analisi Comparativa dei Modelli di Esposizione in Rete	8
2.2.2 Criteri di Selezione e Giustificazione Architettureale	9
2.2.3 Enforcing del Paradigma Zero-Trust	9
2.3 Tassonomia delle Minacce e Vulnerabilità Logiche	10
2.3.1 Il Semantic Gap e l'Evoluzione del Panorama delle Minacce	10
2.3.2 Categorie di Fallimento Architettureale	11
2.3.3 Dal Modello Teorico alla Necessità di Valutazione Strutturata	12
2.4 L'Evoluzione del Testing di Sicurezza: dall'Analisi Cieca ai Contratti	12
2.4.1 Paradigmi Tradizionali e i Limiti del DAST/SAST	12
2.4.2 Asimmetria Informativa e il Box Gradient	13
2.4.3 Contract-Driven Security e Approcci Ibridi	14

3	Metodologia, Architettura e Principi di Design	16
3.1	Principi Fondamentali: il Perché delle Scelte Architettureali	16
3.1.1	Agnosticismo Applicativo (D1.P1)	17
3.1.2	La Specifica OpenAPI come Unico Oracolo Formale (D3.P2) . . .	18
3.1.3	Config-Driven Development (D3.P1)	19
3.1.4	Riproducibilità Deterministica (D3.P4)	20
3.2	Architettura dell'Assessment Engine	20
3.2.1	Flusso di Dipendenze Monodirezionale (D1.P2)	21
3.2.2	Scheduling Basato su DAG e Orchestrazione Deterministica (D2.P2)	21
3.2.3	Split State e Immutabilità (D1.P3)	22
3.2.4	Gateway-Agnostic Adapter (D1.P6)	24
3.2.5	Streaming Evidence Store (D4.P1)	24
3.2.6	Assenza di Dipendenze di Stato Esterno (D3.P5)	25
3.3	Domini di Sicurezza e Box Gradient Applicato	25
3.3.1	Tassonomia degli Otto Domini di Assurance	26
3.3.2	Il Box Gradient come Mapping dalle Precondizioni al Privilegio (D3.P3)	27
4	Implementazione	29
4.1	La Pipeline di Esecuzione a Sette Fasi e il Core Engine	29
4.1.1	Fail-Safe Error Isolation (D4.P3)	33
4.1.2	Gerarchia delle Eccezioni con Phase Mapping (D4.P4)	33
4.1.3	Costruzione dei Contesti di Esecuzione (D1.P3)	35
4.2	Discovery Dinamico e Risoluzione del Grafo	35
4.2.1	Discovery Dinamico via <code>pkgutil</code> (D2.P1)	36
4.2.2	Scheduling Topologico e <code>DAGScheduler</code> (D2.P2)	37
4.3	Architettura dei Test Nativi e Contratto <code>BaseTest</code>	38
4.3.1	Gerarchia Duale <code>BaseTest</code> / <code>ExternalToolTest</code> (D1.P4)	38
4.3.2	Invarianti di <code>TestResult</code> a Livello di Tipo (D4.P9)	39
4.3.3	Auth Abstraction Layer e Idempotenza dei Token (D2.P5)	40
4.3.4	Tracciabilità Normativa e Distinzione Finding/InfoNote (D5.P1, D5.P2)	41
4.3.5	Oracle a Tre Esiti e Safe HTTP Probing Policy (D4.P7)	41
4.3.6	Path Seed System (D2.P7)	43
4.3.7	Catalogo dei Payload di Attacco (D2.P8)	43
4.3.8	Teardown Best-Effort in Ordine LIFO (D4.P6)	43
4.4	Gerarchia dei Connettori e Integrazione Tool Esterni	44
4.4.1	Gerarchia a Tre Livelli e Separazione Dati/Valutazione (D1.P5, D2.P3)	45
4.4.2	Lifecycle dei Connector e Dependency Injection (D2.P4)	46
4.4.3	Classificazione A/B dei Connector (D2.P6)	46
4.4.4	Isolamento della Dipendenza AGPL (D7.P2)	47
4.5	Implementazione del Gateway Adapter	47
4.5.1	<code>KongGatewayAdapter</code> : Accesso Read-Only all'Admin API	47

Indice

4.5.2	Degradazione Controllata a Tre Livelli (D4.P5)	48
4.6	Evidence Store e Sanitizzazione delle Credenziali	49
4.6.1	Ciclo di Vita dello Streaming Evidence Store (D4.P1)	49
4.6.2	Sanitizzazione Centralizzata delle Credenziali (D4.P2)	50
4.6.3	Dual Audit Trail e Report Domain-Centric (D5.P5)	51
5	Validazione sperimentale e risultati	53
5.1	Topologia del Testbed Sperimentale	53
5.1.1	Composizione e Connettività dell'Ambiente di Test	54
5.1.2	Configurazione del Testbed per la Validazione Funzionale	54
5.1.3	Struttura RBAC e Provisioning degli Utenti	56
5.2	Qualità della Codebase come Prerequisito di Credibilità	57
5.2.1	Analisi Statica della Codebase	57
5.2.2	Dual-Layer Type Safety (D6.P3)	58
5.2.3	Indicatori di Qualità dell'Ingegneria di Produzione	59
5.3	Copertura Funzionale e Idempotenza dell'Assessment	60
5.3.1	Analisi dei Risultati per Dominio	61
5.3.2	Idempotenza e Validazione del Teardown	62
5.4	Footprint Computazionale e Benchmarking del DAG	63
5.4.1	Regime di Esecuzione: Dominato dalla Latenza di Rete	63
5.4.2	Topologia del DAG e Implicazioni sull'Ordinamento dell'Esecuzione	64
	Sintesi della Validazione	65
6	Discussione e Sviluppi Futuri	66
6.1	Analisi Critica e Limiti del Paradigma Contract-Driven	66
6.1.1	Il Paradosso del Ground Truth (D3.P2)	66
6.1.2	La Tensione dell'Astrazione (D1.P6)	67
6.1.3	La Dipendenza dall'Admin API (D4.P5)	67
6.2	Applicabilità Industriale e Integrazione DevSecOps	68
6.2.1	Semantic Exit Codes come Canale di Comunicazione con la Pipeline (D6.P1)	68
6.2.2	Quality Gate a Due Velocità: Fail-Fast e Assessment Completo (D4.P8)	69
6.2.3	Variabilità di Privilegio nei Deployment Industriali	70
6.3	Sviluppi Futuri	70
6.3.1	Estensioni Operative	70
6.3.2	Traiettorie di Ricerca Aperta	72
7	Conclusioni	74
7.1	Sintesi del Lavoro e Risoluzione del Problema Iniziale	74
7.2	Contributi Scientifici e Ingegneristici	75
7.3	Considerazioni Finali sull'Assurance Continuo	76
	Bibliografia	79

Elenco delle figure

Figura 4.1 Layout di <code>src/</code> con mapping alle fasi della pipeline.	31
--	----

Elenco delle tabelle

Tabella 3.1	Proprietà APIGuard Assurance per dominio.	17
Tabella 3.2	Domini di Assurance di APIGuard Assurance.	26
Tabella 3.3	Box Gradient applicato in APIGuard Assurance.	27
Tabella 4.1	Fasi della pipeline di assessment (1-2-4 bloccanti; 5-6-7 non-bloccanti).	30
Tabella 4.2	Gerarchia delle eccezioni custom (Fasi 1-4 bloccanti; Fasi 5-6 recupe- rate localmente).	34
Tabella 5.1	Componenti e versioni dell'ambiente di test.	55
Tabella 5.2	Risultati dell'analisi statica su APIGuard Assurance v0.1.0.	58
Tabella 5.3	Sintesi delle verifiche di qualità sulla codebase.	60
Tabella 5.4	KPI di idempotenza su due run indipendenti.	61
Tabella 5.5	Status e finding count per test.	62
Tabella 5.6	Metriche di sistema sui due run.	63
Tabella 5.7	Topologia del Directed Acyclic Graph (DAG) nell'assessment di vali- dazione.	64

Elenco dei codici

Capitolo 1

Introduzione

Le architetture a microservizi cloud-native hanno reso le Application Programming Interface (API) REST il principale confine operativo tra sistemi: ogni servizio espone un contratto programmabile, e la composizione di quei contratti definisce una superficie di attacco che cresce con le interazioni tra servizi più che con il loro numero. Valutare la sicurezza di questa superficie richiede strumenti che conoscano il contratto che ogni richiesta dovrebbe rispettare; gli scanner tradizionali, privi di quella conoscenza, operano sulla sintassi e rimangono ciechi alla classe di vulnerabilità semantiche che caratterizza le API REST. Questo lavoro progetta, implementa e valida empiricamente un framework di security assessment che usa la specifica OpenAPI come oracolo formale per costruire verifiche riproducibili e prioritizzate su qualsiasi sistema REST documentato, senza dipendenze da un target specifico.

1.1 Motivazione e Contesto

L'adozione delle architetture a microservizi cloud-native ha redistribuito la complessità applicativa su un numero crescente di servizi autonomi, ciascuno responsabile di un sottodominio del sistema e comunicante con gli altri tramite interfacce programmabili. La conseguenza diretta è la moltiplicazione della superficie esposta: ogni servizio aggiunge endpoint al perimetro, spesso in assenza di un inventario centralizzato e di una governance sistematica del ciclo di vita delle API. Il fenomeno genera tre categorie di esposizione non intenzionale: le API deprecated che continuano a rispondere per mancata disattivazione, gli endpoint raggiungibili ma assenti da qualsiasi specifica, e le interfacce amministrative esposte per errore di configurazione. Tutte e tre sono invisibili agli strumenti che non dispongono di una rappresentazione formale della superficie esposta: uno scanner che costruisce il proprio modello dal traffico osservato o dalla documentazione disponibile non può rilevare ciò che non è documentato o non ha ancora prodotto traffico. Il Capitolo 2 ne analizza le implicazioni in dettaglio.

L'impatto di questa esposizione sul panorama della sicurezza è documentato da più fonti convergenti. L'Open Web Application Security Project (OWASP) API Security Top 10 [?] formalizza le dieci categorie di rischio più frequenti nelle API REST, con le prime cinque interamente composte da vulnerabilità semantiche: violazioni di autorizzazione a livello di oggetto (Broken Object Level Authorization (BOLA)), difetti di autenticazione, esposizione non intenzionale di proprietà e consumo incontrollato di risorse. Le API hanno consolidato il proprio ruolo di vettore primario negli incidenti di sicurezza enterprise [?], con una frequenza di violazioni correlata a configurazioni errate o a controlli di autorizzazione insufficienti che segnala un disallineamento strutturale tra la velocità con cui le API vengono esposte e la maturità degli strumenti disponibili per valutarne la sicurezza in modo sistematico.

La combinazione di scala e complessità semantica rende il security assessment manuale inadeguato come pratica sistematica: un penetration test condotto su un campione ristretto di endpoint, in un momento specifico, produce un'osservazione puntuale che non è confrontabile con esecuzioni successive e non scala all'intera superficie esposta. La necessità è dotare il processo di assessment di proprietà formali: riproducibilità deterministica dei risultati, tracciabilità dei verdeti alla specifica contrattuale del target, e integrabilità nelle pipeline di sviluppo continuativo senza richiedere intervento manuale a ogni release. Le ragioni per cui gli strumenti esistenti non soddisfano congiuntamente questi requisiti sono analizzate nella Sezione 1.2.

1.2 Definizione del Problema e Gap nella Ricerca

Gli approcci esistenti al security assessment delle API REST presentano tre limitazioni strutturali distinte, ciascuna radicata in una scelta di design diversa. Identificarle separatamente è necessario perché richiedono risposte architetturali diverse.

G₁ Cecità Semantica Il primo gap riguarda la capacità di osservazione. Gli scanner Dynamic Application Security Testing (DAST) e i fuzzer generici operano sulla sintassi delle richieste HyperText Transfer Protocol (HTTP): generano payload, osservano i codici di risposta e cercano pattern anomali nel traffico. Questa strategia è efficace per le vulnerabilità che si manifestano nella forma della request o della response, come le injection sui parametri. Le vulnerabilità semantiche delle API REST appartengono a una classe diversa: si manifestano come richieste sintatticamente corrette che violano le assunzioni implicite del sistema sul chiamante. Una violazione di autorizzazione orizzontale (BOLA) è, dal punto di vista dello scanner, una richiesta GET perfettamente formata verso un endpoint documentato; il problema è che il chiamante accede a una risorsa su cui i propri privilegi sono insufficienti. Un bypass della revoca dei token è una richiesta autenticata con un token che il sistema avrebbe dovuto invalidare; lo scanner,

privo di conoscenza del contratto che ne governa il ciclo di vita, produce una risposta 2xx e la classifica come esito corretto. Il combinatorial wall, discusso nel Capitolo 2, è la conseguenza diretta: i payload che raggiungono la logica semantica dell'API sono quelli costruiti nel rispetto del contratto, e costruirli sistematicamente richiede la conoscenza di quel contratto. La stessa cecità si manifesta a livello di sequenziamento: molte vulnerabilità di autorizzazione emergono solo in sequenze di chiamate con dipendenze di stato (un token prodotto da una chiamata di login usato in una chiamata successiva, o una risorsa creata da un test e revocata da un altro), che uno scanner privo di conoscenza del contratto non può costruire in modo controllato.

G₂ Portabilità e Profondità Semantica Il secondo gap riguarda la portabilità. Gli strumenti specializzati di security assessment per API si distribuiscono agli estremi di uno spettro: gli strumenti vendor-specific, progettati per un gateway o una piattaforma specifica, offrono profondità ma rimangono circoscritti a quella piattaforma; i framework generici, sufficientemente astratti da operare su qualsiasi target, impiegano la specifica OpenAPI come documentazione di riferimento anziché come sorgente autoritativa da cui derivare a runtime l'intera conoscenza del target. Portabilità e profondità semantica vengono trattate come requisiti in tensione, invece di essere riconciliate attraverso la specifica formale come oracolo comune.

G₃ Riproducibilità Deterministica Il terzo gap riguarda la riproducibilità. Il penetration testing tradizionale è per natura point-in-time: condotto da un operatore in un momento specifico, con un insieme specifico di tecniche e conoscenze, il suo output riflette tanto le caratteristiche del target quanto le scelte di chi ha condotto la sessione. Integrare un assessment come quality gate in una pipeline Continuous Integration / Continuous Deployment (CI/CD) richiede che la stessa condizione, valutata in momenti diversi sullo stesso target invariato, produca lo stesso esito; la variabilità introdotta dal fattore umano e dalla non dichiarazione delle dipendenze preclude questa classe di utilizzo. La riproducibilità come invariante verificabile empiricamente su esecuzioni successive, anziché come proprietà enunciata, è uno dei problemi aperti che questo lavoro affronta.

1.3 Obiettivi della Ricerca

Ai tre gap identificati corrispondono tre obiettivi di ricerca, ciascuno formulato attorno a un meccanismo verificabile.

O₁ Framework Contract-Driven Agnostico Il primo obiettivo, in risposta al G₁ Cecità Semantica, è progettare e implementare un framework che utilizzi la specifica OpenAPI come oracolo formale, derivando a runtime tutta la conoscenza del target

dalla specifica stessa e dal file di configurazione. La conoscenza del contratto ha una conseguenza ulteriore rispetto alla sola precisione semantica: un framework che sa quale garanzia sta verificando può collegare ogni verdetto alla categoria OWASP API Security, al codice Common Weakness Enumeration (CWE) e alle sezioni normative pertinenti, producendo evidenza tracciabile anziché un semplice contatore di anomalie. Il principio architetturale che governa questa scelta è discusso nella Sezione 3.1.1; la sua validazione empirica è descritta nel Capitolo 5.

O₂ Profondità Semantica con Portabilità Il secondo obiettivo, in risposta al G₂ Portabilità e Profondità Semantica, è dimostrare per costruzione che portabilità e profondità semantica sono riconciliabili attraverso la specifica formale come oracolo comune. Un framework che costruisce richieste nel rispetto del contratto raggiunge la logica semantica del target in modo sistematico e indipendente dalla piattaforma. Il meccanismo che realizza questa capacità, la specifica OpenAPI come unica sorgente di verità, è descritto nella Sezione 3.1.2.

O₃ Riproducibilità Verificata e Integrabilità CI/CD Il terzo obiettivo, in risposta al G₃ Riproducibilità Deterministica, è garantire la riproducibilità deterministica dell'assessment e validarla empiricamente su esecuzioni successive, producendo un tool integrabile in pipeline CI/CD tramite exit code semantici. La riproducibilità come invariante verificabile, anziché come proprietà dichiarata, è il criterio che distingue un tool integrabile da un assessment point-in-time. Il meccanismo architetturale è descritto nella Sezione 3.1.4; il canale di comunicazione con la pipeline è discusso nella Sezione 6.2.1.

1.4 Organizzazione della Tesi

Il Capitolo 2 stabilisce il background teorico e la tassonomia delle minacce, analizzando i paradigmi delle architetture API moderne, il pattern del gateway come punto di enforcement della sicurezza, e l'evoluzione degli approcci di testing che hanno preceduto quello contract-driven. Il Capitolo 3 presenta la metodologia e le proprietà architetturali fondamentali: stabilisce il perché delle scelte di design prima di descriverne i componenti, operando a livello di sistema. Il Capitolo 4 porta a livello implementativo quanto stabilito nel Capitolo 3, nominando classi, meccanismi Python specifici e la struttura della pipeline di esecuzione a sette fasi. Il Capitolo 5 valida empiricamente il sistema su Forgejo 14 esposto tramite Kong 3.9, riportando copertura funzionale, idempotenza su due run indipendenti e profilo computazionale con dati tracciabili. Il Capitolo 6 esamina i limiti strutturali del paradigma contract-driven, le condizioni di integrabilità industriale e le traiettorie di sviluppo futuro distinte in estensioni operative e direzioni di ricerca. Il Capitolo 7 traccia la chiusura dell'arco argomentativo e colloca il contributo nel contesto della sicurezza come disciplina ingegneristica.

Capitolo 2

Background e Stato dell'Arte

Il problema affrontato in questa tesi non è sorto nel vuoto: è il prodotto di una traiettoria evolutiva precisa, in cui la distribuzione della logica applicativa su reti di servizi autonomi ha moltiplicato la superficie esposta e reso obsoleti gli strumenti pensati per architetture monolitiche. Questo capitolo ricostruisce quella traiettoria in quattro movimenti. La Sezione 2.1 analizza i paradigmi architetturali e i protocolli di comunicazione che definiscono il perimetro delle API moderne. La Sezione 2.2 esamina il pattern dell'API gateway come punto di enforcement centralizzato della sicurezza nel paradigma Zero-Trust. La Sezione 2.3 classifica le categorie di minaccia rilevanti per le API REST esposte tramite gateway. La Sezione 2.4 analizza l'evoluzione degli approcci di testing, identificando i gap strutturali che motivano l'approccio contract-driven adottato nei capitoli successivi.

2.1 Paradigmi delle Architetture API Moderne

2.1.1 Evoluzione dei Sistemi Distribuiti

L'architettura monolitica, in cui tutta la logica applicativa risiede in un singolo processo deployabile, ha dominato lo sviluppo enterprise per decenni. Le sue proprietà sono note: deployment atomico, transazioni interne semplificate, toolchain di sviluppo consolidato. Il suo limite strutturale lo è altrettanto: la scala è ottenuta replicando l'intero monolite, indipendentemente da quale sottoinsieme funzionale sia effettivamente sotto carico, e ogni modifica richiede il redeployment dell'intera applicazione, aumentando il rischio di regressione con la crescita della codebase.

L'architettura a microservizi risponde a questo limite decomponendo il sistema in servizi autonomi, ciascuno responsabile di un sottodominio delimitato del business, deployabile e scalabile indipendentemente dagli altri [?]. La decomposizione ridistribuisce la complessità: ogni servizio è più semplice del monolite, ma il sistema nel suo complesso introduce

un nuovo ordine di problemi che il monolite non presentava. La comunicazione tra servizi avviene attraverso interfacce programmabili esposte in rete, ciascuna con il proprio contratto, il proprio ciclo di vita e la propria superficie di attacco. Il numero di queste interfacce cresce con le interazioni tra servizi, non linearmente con il numero di servizi in sé: un sistema a n servizi può esporre fino a $O(n^2)$ canali di comunicazione, ciascuno potenzialmente raggiungibile, interrogabile e abusabile indipendentemente dal contesto applicativo.

2.1.2 Tassonomia degli Stili Architetture e di Comunicazione

I servizi autonomi comunicano attraverso stili architetturali con proprietà tecniche distinte, ciascuno con implicazioni di sicurezza proprie. La rassegna che segue è orientata a quelle proprietà, non alla sintassi dei protocolli.

Representational State Transfer (REST), formalizzato da Fielding nella sua dissertazione del 2000 [?], è uno stile architetturale stateless basato su risorse identificate da Uniform Resource Identifier (URI) e manipolate tramite verbi HTTP standard. La sua proprietà fondamentale è la separazione tra l'identificazione della risorsa e la sua rappresentazione: il client specifica quale risorsa vuole e quale operazione eseguire, il server decide come restituirla. Questa separazione genera una conseguenza di sicurezza strutturale: l'identificatore della risorsa è visibile nel path dell'Uniform Resource Locator (URL) e controllabile dal client, rendendo la verifica dell'autorizzazione a livello di oggetto una responsabilità applicativa che il protocollo non può enforce autonomamente.

Google Remote Procedure Calls (gRPC) adotta un modello diverso: le operazioni sono procedure nominate, i parametri sono tipizzati tramite Protocol Buffers, la serializzazione è binaria e il trasporto è HTTP/2. La tipizzazione forte e la serializzazione binaria chiudono la superficie a certi vettori di injection, ma introducono opacità al livello di ispezione del payload: un Web Application Firewall (WAF) che opera sulla semantica delle richieste HTTP/1.1 è strutturalmente cieco ai frame Protobuf che attraversano un tunnel HTTP/2.

Graph Query Language (GraphQL) espone un singolo endpoint che accetta query arbitrarie sul grafo dello schema: il client specifica esattamente i campi che vuole, compresi i percorsi di attraversamento del grafo. Questa flessibilità sposta il controllo dalla selezione degli endpoint alla validazione della complessità della query: una singola richiesta HTTP può contenere centinaia di query alias, consentendo di aggirare rate limiting configurato a livello di conteggio delle richieste HTTP.

Simple Object Access Protocol (SOAP), protocollo basato su Extensible Markup Language (XML) e contratti Web Services Description Language (WSDL), è prevalente in sistemi enterprise legacy. Le sue vulnerabilità strutturali derivano dal parsing XML: XML

External Entity (XXE) injection e XML Bomb sono conseguenze dirette del modello di elaborazione del documento, indipendenti dall'applicazione specifica.

WebSocket e Server-Sent Events (SSE) condividono la proprietà di stabilire connessioni persistenti che sopravvivono al singolo ciclo request-response. Il rate limiting applicato sull'handshake HTTP iniziale non si estende ai frame successivi, e i meccanismi di ispezione del gateway operano esclusivamente sul traffico di upgrade, lasciando il canale post-handshake fuori dal perimetro di enforcement.

Questa tesi si focalizza su API REST documentate tramite specifica OpenAPI: è lo stile architetturale più adottato nelle architetture a microservizi cloud-native e quello per cui esiste una specifica formale sfruttabile come oracolo di assessment.

2.1.3 Il Fenomeno dell'API Sprawl

La decomposizione in microservizi, combinata con cicli di sviluppo rapidi e team decentralizzati, produce un effetto collaterale documentato: la proliferazione incontrollata di endpoint esposti che sfugge a qualsiasi inventario centralizzato [?]. Il fenomeno si manifesta in tre categorie distinte. Le *Shadow API* sono endpoint funzionanti ma assenti da qualsiasi specifica pubblicata: creati durante lo sviluppo, mai rimossi, raggiungibili ma non documentati. Le *Deprecated API* sono endpoint formalmente obsoleti che continuano a rispondere perché la loro disattivazione non è mai avvenuta o perché dipendenze non mappate li mantengono attivi. Le *Zombie API* sono endpoint di versioni precedenti del servizio che persistono in deployment non aggiornati.

Tutte e tre le categorie condividono una proprietà critica per il security assessment: sono invisibili agli strumenti che costruiscono la propria conoscenza del target a partire dalla documentazione. Un fuzzer che enumera gli endpoint dalla specifica OpenAPI non può raggiungere un endpoint non documentato; uno scanner che confronta le risposte con i codici attesi dalla specifica non può rilevare una Shadow API perché non sa che dovrebbe cercarvela. La rilevazione delle Shadow API richiede un approccio inverso: confrontare la superficie effettivamente raggiungibile con quella dichiarata, identificando la discrepanza. Questo è precisamente uno degli obiettivi del Domain-0 (API Discovery) descritto nella Sezione 3.3.1.

2.2 Il Pattern dell'API Gateway in Ottica Zero-Trust

2.2.1 Analisi Comparativa dei Modelli di Esposizione in Rete

Le architetture a microservizi dispongono di quattro modelli principali per esporre i propri servizi al traffico esterno e per governare il traffico interno, ciascuno con un diverso posizionamento nel perimetro di sicurezza.

L'API gateway enterprise è il punto di terminazione centralizzato per il traffico nord-sud, ovvero il traffico proveniente da client esterni. Concentra in un unico strato le funzioni trasversali che altrimenti andrebbero replicate in ogni servizio: autenticazione e autorizzazione, rate limiting, trasformazione dei payload, terminazione Transport Layer Security (TLS) e logging delle transazioni. La centralizzazione rende il gateway il punto naturale in cui enforcare le policy di sicurezza prima che le richieste raggiungano la logica applicativa; è anche il punto in cui un errore di configurazione produce conseguenze sistemiche, non localizzate a un singolo servizio.

Il modello Kubernetes Ingress e Gateway API governa il traffico nord-sud all'interno di un cluster containerizzato. La risorsa Ingress definisce regole di routing dichiarative (hostname, path, servizio di destinazione) che un Ingress Controller traduce in configurazione del reverse proxy sottostante. La Kubernetes Gateway API, introdotta come standard stabile nel 2023, risolve le limitazioni di portabilità del modello Ingress sostituendo le annotation controller-specific con risorse tipizzate e role-oriented. Dal punto di vista del security assessment, il gateway gestisce il traffico di upgrade e l'autenticazione iniziale, ma la sua capacità di ispezione si limita al piano HTTP: traffico WebSocket post-handshake e payload Protobuf attraversano il layer senza ispezione del contenuto.

Il service mesh governa il traffico est-ovest, ovvero le comunicazioni inter-servizio all'interno del perimetro. Architetture basate su sidecar proxy (Envoy nel caso di Istio) intercettano tutto il traffico del pod in modo trasparente, abilitando mutua autenticazione TLS (mTLS), osservabilità e circuit breaking senza modifiche al codice applicativo. La mesh introduce però assunzioni implicite sulla natura del traffico: un sidecar che bufferizza messaggi per implementare retry e metriche assume che i flussi siano finiti e di durata limitata; uno stream gRPC bidirezionale indefinito viola questa assunzione e può produrre esaurimento di memoria nel proxy.

Il modello serverless delega la gestione del ciclo di vita dei container alla piattaforma cloud. Il traffico è gestito da un gateway managed che scala automaticamente e non richiede configurazione infrastrutturale esplicita. Il costo di questa astrazione è visibile sul piano della sicurezza: l'accesso amministrativo al piano di controllo è mediato da policy IAM della piattaforma e non è disponibile nella forma di un'Admin API ispezionabile, il che rende inapplicabili le verifiche `WHITE_BOX` descritte nella Sezione 3.3.2.

2.2.2 Criteri di Selezione e Giustificazione Architetturale

Dei quattro modelli analizzati, l'API gateway enterprise è l'unico che soddisfa congiuntamente i requisiti del framework proposto in questa tesi. Tre criteri guidano questa selezione.

Il primo criterio è la verificabilità del piano di controllo. Un gateway enterprise espone un'Admin API che consente l'ispezione diretta della configurazione: plugin attivi, policy di routing, parametri di rate limiting, variabili d'ambiente. Questa interfaccia è il presupposto tecnico dei test `WHITE_BOX`, che verificano garanzie non osservabili dall'esterno tramite probe empirici. Il service mesh delega parte di questa visibilità al control plane distribuito; il modello serverless la sottrae interamente.

Il secondo criterio è la separazione tra piano di autenticazione e logica applicativa. Un gateway che centralizza l'enforcement delle policy di autenticazione e autorizzazione consente di valutare questi meccanismi indipendentemente dall'applicazione ospitata: un test che verifica il rifiuto delle richieste non autenticate non deve conoscere la logica applicativa, deve conoscere solo il contratto che il gateway dovrebbe enforcare. Questa separazione è la condizione che rende il framework applicazione-agnostico nel senso stabilito nella Sezione 3.1.1.

Il terzo criterio è la documentazione formale delle API esposte. Un gateway enterprise instrada il traffico verso applicazioni che espongono la propria interfaccia tramite specifica OpenAPI; questa specifica è la sorgente da cui il framework deriva a runtime tutta la conoscenza del target. I modelli serverless e mesh non escludono questa possibilità, ma non la presuppongono come requisito architetturale.

2.2.3 Enforcing del Paradigma Zero-Trust

Il paradigma Zero-Trust, formalizzato nel NIST Special Publication 800-207 [?], ridefinisce il perimetro di sicurezza spostando il punto di controllo dalla rete all'identità: nessuna richiesta è considerata attendibile per via della sua provenienza da una rete ritenuta fidata, e ogni accesso è valutato in modo indipendente rispetto all'identità del chiamante, al contesto della richiesta e ai permessi sulla risorsa specifica. Applicato alle API REST, il principio si traduce in tre requisiti operativi: autenticazione obbligatoria su ogni endpoint, autorizzazione valutata per ogni coppia chiamante-risorsa, e assenza di fiducia implicita derivante dalla posizione di rete del client.

L'API gateway è il punto architetturale in cui questi requisiti trovano enforcement naturale: si colloca tra il client e il backend, ha visibilità su ogni richiesta in transito, e può rifiutare le richieste che non soddisfano i controlli configurati prima che raggiungano la logica applicativa. La verifica che questo enforcement sia effettivamente attivo e

correttamente configurato è esattamente il perimetro del framework proposto: le garanzie che il gateway dichiara nella propria configurazione (autenticazione richiesta su tutti gli endpoint, rate limiting attivo, header di sicurezza iniettati) devono corrispondere al comportamento osservabile a runtime, e questa corrispondenza non è verificabile senza un assessment sistematico. La distanza tra configurazione dichiarata e comportamento effettivo è il gap che motiva il dominio Configuration & Hardening (Domain-6) e i test WHITE_BOX della Sezione 3.3.2.

2.3 Tassonomia delle Minacce e Vulnerabilità Logiche

2.3.1 Il Semantic Gap e l'Evoluzione del Panorama delle Minacce

La transizione verso architetture a microservizi ha spostato il baricentro delle vulnerabilità dalle anomalie sintattiche alle violazioni semantiche. Un WAF tradizionale intercetta una SQL injection perché riconosce nel payload un pattern sintattico noto; la stessa infrastruttura è strutturalmente cieca a una richiesta GET /users/456 perfettamente formata inviata da un utente che ha il permesso di leggere solo /users/123, perché quella violazione non si esprime nella forma della richiesta ma nella relazione tra l'identità del chiamante e la risorsa richiesta. Il WAF ispeziona il protocollo; la vulnerabilità risiede nel contratto applicativo.

Questa distanza tra ciò che uno strumento di testing può osservare e ciò che una vulnerabilità semantica richiede di comprendere costituisce il *semantic gap*: il divario strutturale tra la forma della transazione HTTP e la semantica del contratto che quella transazione dovrebbe rispettare. Il framework OWASP API Security Top 10 nella sua edizione 2023 [?] formalizza questa transizione: le prime cinque categorie di rischio, Broken Object Level Authorization (API1), Broken Authentication (API2), Broken Object Property Level Authorization (API3), Unrestricted Resource Consumption (API4) e Broken Function Level Authorization (API5), sono tutte vulnerabilità semantiche. La loro manifestazione comune è una richiesta sintatticamente corretta che viola le assunzioni implicite del sistema sull'identità, i privilegi o il comportamento atteso del chiamante.

Colmare il semantic gap richiede che lo strumento di assessment possieda una rappresentazione formale del contratto API e la utilizzi come oracolo per valutare le risposte ricevute. La specifica OpenAPI è quella rappresentazione: definisce quali endpoint esistono, quali parametri accettano, quali schemi di risposta producono e quali codici di stato sono attesi. Uno strumento che costruisce le proprie richieste nel rispetto di questa specifica e valuta le risposte rispetto a ciò che il contratto prevede per quella specifica operazione è in grado di raggiungere la classe di vulnerabilità che i fuzzer sintattici lasciano intatta. Questa è la premessa teorica dell'approccio adottato, il cui meccanismo implementativo è descritto nella Sezione 3.1.2.

2.3.2 Categorie di Fallimento Architetturale

Le vulnerabilità semantiche delle API REST si raggruppano in tre macro-categorie distinte per natura del fallimento, in ordine crescente di complessità del vettore di attacco.

La prima categoria racchiude i difetti di autorizzazione e gestione dell'accesso agli oggetti. Il Broken Object Level Authorization (BOLA, API1:2023) si manifesta quando un sistema verifica che il chiamante sia autenticato ma non verifica che abbia il diritto di accedere alla specifica risorsa identificata dal path. La richiesta è formalmente corretta; il fallimento risiede nell'assenza o nell'inadeguatezza del controllo sulla coppia chiamante-oggetto. Il Broken Function Level Authorization (Broken Function Level Authorization (BFLA), API5:2023) segue la stessa logica a livello di operazione: un utente con un ruolo ristretto riesce a invocare un endpoint riservato a ruoli privilegiati, tipicamente perché i controlli di autorizzazione sono applicati in modo non uniforme tra endpoint dello stesso servizio. Entrambe le classi richiedono, per essere rilevate in modo sistematico, la capacità di effettuare richieste con credenziali di ruoli distinti e confrontarne le risposte: un workflow multi-step che i fuzzer stateless non supportano.

La seconda categoria comprende le violazioni del principio di minimo privilegio nel flusso dei dati. Il mass assignment si verifica quando un endpoint accetta nel payload di richiesta campi non documentati nello schema contrattuale, che il backend mappa direttamente sul modello di dominio senza whitelist esplicita: un attaccante può elevare i propri privilegi o modificare attributi riservati semplicemente includendo campi aggiuntivi in un POST o PUT. L'esposizione eccessiva di dati (API3:2023) è la controparte nella risposta: il servizio restituisce campi sensibili, tra cui hash di password, chiavi di API o dati personali identificativi, perché il livello applicativo serializza l'intero oggetto di dominio affidando al client la selezione dei campi rilevanti. Entrambi i fallimenti sono invisibili a uno strumento che non conosce lo schema contrattuale atteso: a livello HTTP, la transazione appare corretta in entrambi i sensi.

La terza categoria abbraccia l'abuso della topologia distribuita come vettore di attacco indiretto. Il Server-Side Request Forgery (SSRF) (API7:2023) sfrutta endpoint che accettano URL forniti dal client, come webhook, mirror o proxy, per far sì che il server effettui richieste verso infrastrutture interne o verso i metadata endpoint dei cloud provider, esfiltrando credenziali di esecuzione o raggiungendo servizi non esposti al perimetro esterno. L'assenza di rate limiting (API4:2023) espone il sistema all'esaurimento controllato delle risorse: un attaccante che può inviare richieste senza limitazione può saturare il backend, i pool di connessioni al database o la capacità computazionale, rendendo il servizio indisponibile per gli utenti legittimi. Entrambe le classi operano attraverso funzionalità legittime del sistema, non attraverso payload malformati, e sono quindi sistematicamente invisibili ai layer di ispezione sintattica.

2.3.3 Dal Modello Teorico alla Necessità di Valutazione Strutturata

Le tre categorie identificate nella sezione precedente non sono omogenee né per il tipo di condizione che richiedono né per il meccanismo di rilevazione che presuppongono. Rilevare una violazione BOLA richiede token validi per almeno due identità distinte e la capacità di confrontare le risposte ricevute con credenziali diverse. Rilevare mass assignment richiede di conoscere lo schema di richiesta contrattuale e di verificare il comportamento del sistema in presenza di campi non dichiarati. Rilevare un SSRF richiede endpoint che accettino URL e un meccanismo per osservare le richieste che il server effettua verso l'esterno. Nessuno dei tre fallimenti è raggiungibile con lo stesso strumento e la stessa configurazione.

Questa eterogeneità è la ragione per cui un singolo scanner generico non è sufficiente: le classi di vulnerabilità semantica impongono condizioni di accesso distinte, richiedono strategie di osservazione diverse e non si mappano sulla stessa superficie di attacco. La risposta architetturale a questa eterogeneità è la tassonomia a otto domini di assurance descritta nella Sezione 3.3.1, in cui ogni dominio raggruppa le garanzie che condividono le stesse condizioni informative e lo stesso tipo di evidenza necessario per la verifica. Il box gradient, discusso nella Sezione 3.3.2, formalizza come il livello di visibilità disponibile determini la classe di garanzie raggiungibile: la struttura di condizioni che emerge dall'analisi delle minacce in questa sezione non è un ostacolo da aggirare, ma il criterio che orienta la classificazione dei test nel Capitolo 3.

2.4 L'Evoluzione del Testing di Sicurezza: dall'Analisi Cieca ai Contratti

2.4.1 Paradigmi Tradizionali e i Limiti del DAST/SAST

Gli approcci classici al security testing delle applicazioni web si dividono in due famiglie distinte per punto di osservazione. Il Static Application Security Testing (SAST) analizza il codice sorgente o i binari compilati senza eseguire l'applicazione, identificando pattern noti di vulnerabilità a livello sintattico: buffer overflow, uso di funzioni non sicure, flussi di dati non validati verso sink pericolosi [?]. Il DAST interagisce con l'applicazione in esecuzione attraverso la sua interfaccia esterna, inviando richieste e osservando le risposte alla ricerca di comportamenti anomali: codici di errore rivelatori, stack trace nelle risposte, variazioni nei tempi di risposta correlate a payload specifici.

Entrambi i paradigmi, applicati alle API REST, incontrano un limite strutturale prima ancora di raggiungere la logica applicativa. Il DAST genera payload a partire da euristiche sintattiche: variazioni di parametri, payload di injection, valori ai limiti degli intervalli

attesi. Questi payload vengono costruiti senza conoscere il contratto che l'endpoint dichiara; di conseguenza, una quota sostanziale delle richieste viola i tipi, i formati o i vincoli enumerati nello schema e viene respinta con HTTP 400 dal layer di validazione dell'input, prima che qualsiasi logica di business venga valutata. Questo fenomeno, denominato *combinatorial wall*, è la barriera strutturale che impedisce al fuzzing cieco di raggiungere sistematicamente la classe di vulnerabilità semantica descritta nella Sezione 2.3.2: lo spazio di tutti i payload sintatticamente possibili è astronomico, ma lo spazio dei payload strutturalmente validi rispetto al contratto è definito e navigabile, e soltanto quest'ultimo è in grado di interrogare la logica applicativa. Il SAST, dal canto suo, analizza il codice ma non può osservare il comportamento a runtime in presenza di identità autenticate con ruoli distinti: una violazione BOLA emerge dall'interazione tra credenziali, routing del gateway e logica di autorizzazione, non da un pattern sintattico nel codice sorgente isolato.

Lo strumento DAST più diffuso nel contesto delle API, OWASP ZAP [?], è progettato per operare su applicazioni web generiche e supporta API REST tramite importazione della specifica OpenAPI. Anche in questa modalità, tuttavia, ZAP utilizza la specifica per enumerare gli endpoint e costruire richieste valide, ma valuta le risposte rispetto a euristiche generiche di anomalia, non rispetto a ciò che il contratto prevede per quella specifica operazione su quella specifica risorsa con quelle specifiche credenziali. La distinzione è sostanziale: un campo sensibile restituito in una risposta 200 è un finding solo se lo schema di risposta contrattuale non lo dichiarava atteso; uno scanner privo di quella conoscenza non può rilevarlo come anomalia.

2.4.2 Asimmetria Informativa e il Box Gradient

La capacità di rilevare una vulnerabilità non dipende solo dalla sofisticazione dello strumento, ma dalla quantità di informazione che lo strumento possiede sul sistema sotto test prima di iniziare la valutazione. Questa asimmetria informativa è il principio che sottende la classificazione Black/Grey/White Box nel testing del software [?]: un tester che non ha accesso al codice sorgente né alla configurazione interna del sistema opera nel regime Black Box, limitando la propria osservazione al comportamento esterno; un tester con accesso completo alla configurazione interna opera nel regime White Box, potendo verificare per ispezione diretta proprietà non osservabili dall'esterno.

Applicata alle API REST esposte tramite gateway, questa distinzione acquista una precisione tecnica che va oltre la semplice metafora della visibilità. Alcune garanzie di sicurezza sono fisicamente raggiungibili solo in determinati regimi di visibilità: verificare che il gateway rifiuti le richieste non autenticate non richiede nulla più dell'accesso di rete, perché il gateway applica quel controllo a qualsiasi interlocutore; simulare un attaccante esterno privo di credenziali è l'unico modo per ottenere evidenza non contaminata da privilegi impliciti. Verificare una violazione BOLA richiede almeno due identità con ruoli

distinti, entrambe autenticate: senza quello stato il gateway respinge la richiesta prima che la logica di autorizzazione venga valutata, e l'esito è indistinguibile da un corretto rifiuto di autenticazione. Verificare che le variabili d'ambiente del gateway non contengano credenziali in chiaro richiede accesso al piano di configurazione amministrativo: nessun probe empirico verso gli endpoint esposti può restituire quella informazione.

Ne deriva che il gradiente Black/Grey/White Box, nell'approccio adottato in questa tesi, non è una tassonomia teorica applicata retroattivamente ai test, ma il prodotto delle precondizioni concrete che ciascuna garanzia di sicurezza impone al tester. La sua formalizzazione come proprietà architetturale (descritta nella Sezione 3.3.2) è la conseguenza diretta di questa analisi: assegnare a un test un livello di visibilità senza derivarlo dalle sue precondizioni produce una classificazione decorativa, non operativa.

2.4.3 Contract-Driven Security e Approcci Ibridi

Il limite strutturale del fuzzing cieco, l'incapacità di costruire sistematicamente richieste valide rispetto al contratto senza possedere il contratto stesso, ha motivato una linea di ricerca che utilizza le specifiche formali delle API come punto di partenza per la generazione dei test. RESTler [?], il primo fuzzer stateful per API REST presentato all'ICSE 2019, analizza la specifica OpenAPI del servizio target e inferisce le dipendenze produttore-consumatore tra i tipi di richiesta: se un endpoint `POST /resources` produce un `resource_id` che un endpoint `GET /resources/{id}` consuma come parametro di path, RESTler genera automaticamente la sequenza nell'ordine corretto. Questo approccio abbatte il combinatorial wall costruendo richieste strutturalmente valide a partire dal contratto e consentendo al fuzzer di raggiungere la logica applicativa che il fuzzing cieco non raggiunge.

RESTler dimostra la praticabilità tecnica del paradigma contract-driven, ma il suo obiettivo primario è la generazione di sequenze di test per trovare crash e comportamenti inattesi, non la verifica sistematica di garanzie di sicurezza specifiche. La specifica OpenAPI viene utilizzata per costruire le richieste, ma le risposte sono valutate rispetto a euristiche di anomalia generali, non rispetto a ciò che il contratto prevede per quella combinazione di operazione, risorsa e identità del chiamante. Ne deriva che RESTler è efficace nell'identificare difetti di robustezza e nell'esplorare lo spazio degli stati raggiungibili, ma non è progettato per verificare se una risposta 200 contenga campi che il contratto non prevede, se un token revocato venga ancora accettato, o se un endpoint di configurazione del gateway esponga credenziali non cifrate.

Nella stessa direzione si colloca Schemathesis [?], uno strumento di property-based testing che usa la specifica OpenAPI per generare casi di test con riduzione automatica dei controesempi. Rispetto a RESTler, Schemathesis introduce una semantica di valutazione più ricca, ma rimane orientato alla generazione statistica di richieste piuttosto che alla

verifica sistematica di garanzie di sicurezza specifiche: i verdetti dipendono dalle proprietà definite dall'utente, non da un oracolo derivato dal contratto dell'endpoint. Il gap che rimane aperto è precisamente quello che questa tesi intende colmare: un framework che utilizzi la specifica OpenAPI non solo per costruire richieste valide ma come oracolo formale per valutare la correttezza semantica delle risposte rispetto a garanzie di sicurezza predefinite e prioritizzate, che operi in modo agnostico rispetto al target applicativo specifico, e che garantisca la riproducibilità deterministica dei risultati come requisito verificabile empiricamente. Il Capitolo 3 presenta l'architettura che realizza questi tre obiettivi, a partire dai principi fondamentali che li rendono possibili congiuntamente.

Capitolo 3

Metodologia, Architettura e Principi di Design

APIGuard Assurance è un framework di security assessment automatizzato per API REST esposte tramite gateway, progettato per essere applicazione-agnostico, contract-driven e deterministicamente riproducibile. Nasce dalla necessità di superare la frammentazione degli approcci manuali e target-specifici: strumenti che richiedono di essere riscritti per ogni nuovo target, che producono risultati non confrontabili tra esecuzioni, e che non possono essere integrati in processi di verifica continuativi. Il suo obiettivo è produrre un assessment sistematico, tracciabile e ripetibile, applicabile a qualsiasi API REST documentata senza modifiche al codice.

La presentazione di *APIGuard Assurance* segue un approccio *top-down*: dai principi generali di sistema che ne definiscono il perimetro fino alle scelte implementative che li concretizzano. Architettura e implementazione sono trattate in capitoli distinti perché operano su livelli di astrazione distinti. Il Capitolo 4 mostra come i principi descritti vengono implementati.

Il design di *APIGuard Assurance* è formalizzato in 39 proprietà architetture distribuite su sette categorie; la Tabella 3.1 le raccoglie come mappa di riferimento.¹

3.1 Principi Fondamentali: il Perché delle Scelte Architetture

In coerenza con la progressione *top-down* dichiarata, l'analisi ha inizio dal livello di massima astrazione: le decisioni che definiscono il carattere del tool prima ancora di

¹In grassetto le proprietà trattate in questo capitolo.

Dominio	Proprietà
D1: Architettura & Design	D1.P1, D1.P2, D1.P3 , D1.P4, D1.P5, D1.P6
D2: Estensibilità & Manutenibilità	D2.P1, D2.P2 , D2.P3, D2.P4, D2.P5, D2.P6, D2.P7, D2.P8
D3: Config & Riproducibilità	D3.P1, D3.P2, D3.P3, D3.P4, D3.P5
D4: Robustezza & Sicurezza	D4.P1 , D4.P2, D4.P3, D4.P4, D4.P5, D4.P6, D4.P7, D4.P8, D4.P9
D5: Qualità & Osservabilità	D5.P1, D5.P2, D5.P3, D5.P4, D5.P5
D6: CI/CD & DevEx	D6.P1, D6.P2, D6.P3
D7: Packaging & Deployment	D7.P1, D7.P2, D7.P3

Tabella 3.1: Proprietà APIGuard Assurance per dominio.

descrivere i componenti. Le quattro proprietà discusse in questa sezione costituiscono la struttura portante del sistema: le scelte architetturali trattate nelle sezioni successive si stratificano su di esse, ciascuna risolvendo una frizione specifica all'interno dei vincoli che queste proprietà fondamentali stabiliscono.

3.1.1 Agnosticismo Applicativo (D1.P1)

Quali target applicativi è possibile valutare senza modificare il codice del tool?

Tutta la conoscenza del target viene derivata a runtime da due sole sorgenti: il file `config.yaml`, che raccoglie i parametri operativi del deployment (descritti in dettaglio nella Sezione 3.1.3), e la specifica OpenAPI del target, che fornisce la topologia completa degli endpoint, la definizione dei parametri di path, query e header per ciascuna operazione, gli schemi dei payload di richiesta e risposta, e i codici di stato attesi. Ne consegue un vincolo strutturale preciso: APIGuard Assurance non contiene nessun riferimento hardcoded a path, strutture dati o comportamenti di un'applicazione specifica, e qualsiasi API REST documentata è testabile senza modifiche al codice.

Un test che interroga la superficie di attacco derivata dalla specifica e non individua endpoint pertinenti alle proprie condizioni di esecuzione produce **SKIP**, lo stato previsto per uno scope legittimamente vuoto, distinto dall'**ERROR** che segnalerebbe un malfunzionamento interno al tool. Il principio si estende a qualsiasi requisito non soddisfatto: un tool esterno non installato, la Admin API del gateway non raggiungibile, un insieme di endpoint privi di seed configurato; ciascuna di queste condizioni produce **SKIP** sui test che ne dipendono e lascia proseguire gli altri, a meno che il risultato non costituisca una violazione confermata di un controllo perimetrale fondamentale (come l'assenza totale di autenticazione su tutti gli endpoint esposti), caso in cui la pipeline può essere interrotta esplicitamente tramite configurazione.

La differenza tra uno script target-specifico e un tool di assessment generale non è di complessità, ma di generalità: uno script con path e strutture dati scritti nel codice funziona su quel target e solo su quello (sostituita l'applicazione, va riscritto), mentre un tool che deriva tutta la propria conoscenza dalla specifica formale del target è applicabile a qualsiasi API REST documentata senza modifiche. Questa generalità è precisamente il confine che separa un artefatto di progetto da un contributo riutilizzabile.

Il costo accettato è la dipendenza da una specifica valida. Una specifica obsoleta o deliberatamente incompleta produce un assessment parziale senza che il sistema possa rilevarne l'incoerenza: il tool non ha modo di distinguere un'assenza di vulnerabilità da un'assenza di copertura, perché l'unico oracolo di cui dispone è la specifica stessa. Questa tensione costituisce il limite strutturale discusso nella Sezione 6.1 e si qualifica come problema aperto nel testing contract-driven, non come difetto correggibile con intervento locale.

3.1.2 La Specifica OpenAPI come Unico Oracolo Formale (D3.P2)

Quale strumento formale governa la costruzione delle richieste e la valutazione delle risposte nell'intera pipeline?

La specifica OpenAPI è l'unico vincolo contrattuale che governa l'intera pipeline: costruisce ogni richiesta nel rispetto dei tipi, dei formati e dei parametri dichiarati, delimita la superficie di attacco agli endpoint documentati e valuta ogni risposta rispetto a ciò che il contratto prevede per quella specifica operazione.

Un fuzzer che genera payload senza conoscere il contratto affronta un ostacolo strutturale prima ancora di raggiungere la logica applicativa: una quota sostanziale delle richieste viene respinta con HTTP 400 dal layer di validazione dell'input, perché i tipi, i formati e i vincoli enumerati nello schema non sono rispettati. Questo è il *combinatorial wall* discusso nel Capitolo 2: lo spazio di tutti i payload sintatticamente possibili è vastissimo, ma lo spazio dei payload strutturalmente validi rispetto al contratto è definito e navigabile. Usare la specifica come oracolo formale abbatte quella barriera: ogni richiesta è costruita nel rispetto dei vincoli dichiarati, spostando il punto di osservazione dalla sintassi alla semantica.

Ne derivano due capacità operativamente distinte rispetto al fuzzing cieco. In primo luogo, la superficie di attacco viene delimitata con precisione: gli endpoint documentati dalla specifica costituiscono l'insieme di ciò che il contratto dichiara raggiungibile, mentre qualsiasi endpoint risponda senza essere incluso in quella lista è, per definizione, una Shadow API. In secondo luogo, le risposte ricevute vengono valutate rispetto a ciò che il contratto prevede per quella richiesta, non rispetto a euristiche generiche: un campo

sensibile restituito in risposta a una richiesta autenticata è un finding solo se lo schema di risposta contrattuale non lo dichiarava atteso.

Anche questa proprietà porta con sé una dipendenza dalla qualità della specifica, già enunciata per la Sezione 3.1.1, ma con una conseguenza distinta. Lì il problema era di copertura: una specifica incompleta riduce gli endpoint osservabili senza che il tool possa rilevarlo. Qui il problema è di affidabilità dei verdeti: una specifica contrattualmente errata produce valutazioni delle risposte basate su un oracolo sbagliato, e il tool non ha modo di distinguere un contratto corretto da uno incorretto. Si tratta del paradosso del Ground Truth, discusso nella Sezione 6.1 come limite strutturale del paradigma contract-driven e non come anomalia specifica di questa implementazione.

3.1.3 Config-Driven Development (D3.P1)

Esiste un'unica fonte di verità per tutti i parametri che governano il comportamento del tool?

Stabilito che la conoscenza del target risiede nella specifica OpenAPI, rimane da definire il meccanismo che governa l'esecuzione. Tutti i parametri operativi risiedono nel file `config.yaml`, strutturato in aree funzionali con responsabilità distinte: la localizzazione del gateway e della specifica OpenAPI; i parametri di esecuzione globali, tra cui soglia di priorità minima, strategia di filtraggio dei test, timeout per singola esecuzione e formato del logging; il dizionario `path_seed`, che associa a ogni parametro di path parametrico un valore concreto presente sul target; le configurazioni specifiche per singolo test, organizzate per dominio; e i parametri degli strumenti di analisi esterni, con abilitazione per tool, versione attesa e opzioni di invocazione. Le credenziali di autenticazione, per loro natura non versionabili, risiedono in un file `.env` separato, il cui contenuto viene interpolato dal loader prima del parsing. Il codice sorgente non contiene nessun literal decisionale hardcoded: qualsiasi parametro che possa variare tra un'esecuzione e l'altra è dichiarato esplicitamente e validato prima dell'avvio della pipeline.

La separazione tra logica e configurazione assegna a ogni variazione di comportamento un'unica origine dichiarata: il file `config.yaml` attivo durante quella run. Se il comportamento del tool cambia tra un'esecuzione e l'altra, è perché è cambiata la configurazione; se la configurazione è identica, l'output è identico. Questa centralizzazione è anche la condizione che rende possibile la proprietà discussa nella sottosezione seguente: un sistema che legga valori da variabili d'ambiente non dichiarate, da file di stato impliciti o da sorgenti runtime non controllate non può garantire che due esecuzioni successive producano lo stesso risultato, indipendentemente dalla correttezza della logica di test. Fissare la configurazione è il primo passo necessario per fissare l'output.

3.1.4 Riproducibilità Deterministica (D3.P4)

Come viene garantito che l'assessment produca risultati confrontabili tra esecuzioni successive?

Lo stesso target, nella stessa configurazione, interrogato con lo stesso `config.yaml`, deve produrre risultati identici a ogni esecuzione. La riproducibilità è il presupposto che conferisce valore dimostrativo ai risultati del Capitolo 5: senza di essa, ogni run è un'osservazione singola non confrontabile, e l'assessment non è verificabile da terzi.

Il determinismo è ottenuto attraverso tre meccanismi distinti che si completano a vicenda:

- Sezione 3.1.3 (Config-Driven Development): se tutti i parametri decisionali sono fissi, la pipeline non ha sorgenti di variazione endogene.
- Struttura del DAG, trattata nella Sezione 3.2.2: un grafo aciclico orientato con risoluzione topologica deterministica produce sempre la stessa sequenza di batch, indipendentemente dall'ordine in cui i test vengono scoperti a runtime.
- Ordinamento lessicografico dei test all'interno di ogni batch del DAG: l'ordinamento topologico puro non garantisce stabilità tra test pronti allo stesso livello, e senza un criterio di tie-breaking esplicito la sequenza potrebbe variare tra esecuzioni; l'ordine lessicografico risolve questa ambiguità, garantendo che la sequenza sia sempre identica a parità di input.

La garanzia di riproducibilità non riguarda ogni componente dell'output: timestamp e identificatori univoci dei singoli record di evidenza variano per natura tra esecuzioni. Ciò che deve essere invariante sono i contatori aggregati (PASS, FAIL, SKIP), i verdetti per singolo test e la distribuzione dei finding per dominio. La validazione empirica, condotta su due run indipendenti che hanno prodotto contatori aggregati byte-equivalenti (9 PASS / 7 FAIL / 2 SKIP / 0 ERROR / 98 finding, con `diff status per-test = 0` e delta wall-clock inferiore allo 0,3%), è discussa nella Sezione 5.3.

3.2 Architettura dell'Assessment Engine

Le proprietà discusse nella sezione precedente costituiscono la base del sistema. Le scelte di questa sezione si aggiungono ad esse, ciascuna rispondendo a un'esigenza architetturale specifica che le proprietà fondamentali lasciano aperta.

3.2.1 Flusso di Dipendenze Monodirezionale (D1.P2)

Il grafo delle dipendenze tra strati è strettamente aciclico e orientato in una sola direzione: il nucleo del sistema è importato dagli strati di integrazione, che sono importati dagli strati di test, che sono importati dall'orchestratore. Nessun modulo può importare da un livello superiore nella gerarchia. In termini operativi: modificare un test non può rompere il nucleo; aggiungere un connector non può influenzare l'orchestratore; l'intera base del nucleo può essere testata unitariamente senza istanziare alcun client HTTP né alcun test di sicurezza.

In assenza di un vincolo esplicito sulla direzione delle dipendenze, la pressione evolutiva naturale del codice tende verso cicli: un test che trova conveniente importare un'utility da un altro test, un connector che ha bisogno di un modello definito nell'orchestratore, e così via. La direzione è unidirezionale: un test può dipendere dal nucleo per accedere ai componenti condivisi, ma nessun modulo del nucleo può dipendere da un test. Ogni ciclo rende un modulo impossibile da isolare, e un modulo non isolabile non è testabile né sostituibile. Il vincolo è verificabile per ispezione del grafo di importazione: nessun modulo dello strato di test importa componenti degli strati superiori, e ogni strato dichiara esplicitamente nel proprio contratto le sole dipendenze che gli sono ammesse.

La struttura a strati rende praticabile uno degli sviluppi futuri descritti nella Sezione 6.3: un modulo di test contribuito da terze parti viene scoperto automaticamente² senza che nessuna modifica al nucleo del sistema sia necessaria, perché il nucleo non contiene nessun riferimento ai test che lo utilizzano.

3.2.2 Scheduling Basato su DAG e Orchestrazione Deterministica (D2.P2)

L'ordine di esecuzione dei test è calcolato a partire dalle dipendenze che ogni test dichiara esplicitamente come parte del proprio contratto. Lo scheduler raccoglie queste dichiarazioni, costruisce un DAG orientato e risolve l'ordine topologico, producendo una sequenza di batch: gruppi di test il cui insieme di dipendenze è già stato soddisfatto. I batch sono sequenziali tra loro; all'interno di ogni batch i test sono privi di dipendenze reciproche. La traduzione di questo schema in codice, incluse le strutture dati e le librerie impiegate, è descritta nella Sezione 4.2.2.

Il problema che questa struttura risolve non è solo di ordinamento, ma di correttezza semantica. Un test che verifica la validità crittografica di un token JSON Web Token

²La scoperta automatica non è priva di vincoli: un modulo viene incluso nella pipeline solo se rispetta i contratti del tool, ovvero se dichiara tutti gli attributi obbligatori e si colloca nella directory attesa. Un modulo che non soddisfi questi requisiti viene ignorato silenziosamente oppure produce un errore di validazione.

(JWT) presuppone che il meccanismo di autenticazione funzioni: se eseguito prima del test di autenticazione, il token non è disponibile nel contesto condiviso e il test fallirebbe per la ragione sbagliata. Senza una dichiarazione esplicita della dipendenza, la disponibilità del token è una preconditione implicita che dipende dall'ordine di esecuzione scelto: un cambio di scheduling la viola silenziosamente. Con la dipendenza dichiarata, il DAG verifica la preconditione a monte dell'esecuzione, e un token non disponibile diventa un errore rilevabile prima che un singolo test venga avviato.

Lo scheduler include due proprietà di correttezza strutturale:

- Rilevazione statica dei cicli: se il grafo delle dipendenze contiene un ciclo, l'errore viene segnalato prima che un singolo test venga eseguito, impedendo che stati inconsistenti si propaghino silenziosamente nella pipeline.
- Salvaguardia contro lo stallo: se il grafo risulta ancora attivo ma non riesce a estrarre nessun test pronto, il sistema emette un errore strutturato che elenca i test mai schedati e interrompe l'esecuzione in modo controllato, fornendo all'operatore la diagnostica necessaria senza dover ricostruire lo stato interno del grafo.

L'ordinamento topologico puro non garantisce un ordine unico tra test pronti allo stesso livello: senza un criterio di tie-breaking³ esplicito la sequenza potrebbe variare tra esecuzioni. Lo scheduler risolve questa ambiguità ordinando lessicograficamente i test all'interno di ogni batch, convertendo la struttura parzialmente ordinata del grafo in una sequenza riproducibile, condizione necessaria per la proprietà di riproducibilità descritta nella Sezione 3.1.4.

La struttura a batch porta con sé una proprietà che la versione attuale non sfrutta: i test all'interno di uno stesso batch non hanno dipendenze reciproche e sono concettualmente parallelizzabili. Introdurre esecuzione parallela all'interno di un batch è una modifica localizzata allo scheduler, senza impatto sul design dei singoli test, a condizione che le implicazioni di concorrenza sullo stato condiviso vengano analizzate formalmente. Questa traiettoria è documentata come sviluppo futuro nella Sezione 6.3; la sequenzialità attuale è una scelta deliberata di scope, non un vincolo architetturale.

3.2.3 Split State e Immutabilità (D1.P3)

Durante l'esecuzione sequenziale dei test, due categorie di informazione coesistono nel processo: ciò che si sapeva del target prima che la pipeline iniziasse, e ciò che si scopre o si produce durante l'esecuzione. Trattarle con lo stesso oggetto mutabile introduce una categoria di bug difficile da diagnosticare: un test che scrive accidentalmente sulla

³*Tie-breaking*: regola di ordinamento applicata tra elementi a parità di livello topologico, per garantire una sequenza stabile e riproducibile tra esecuzioni.

rappresentazione del target altera la vista condivisa da cui ogni verdetto dipende, senza che il sistema possa rilevarlo.

La soluzione è la separazione netta in due oggetti con caratteristiche opposte:

- **TargetContext** - immutabile per costruzione: viene popolato una volta sola prima che la pipeline inizi e qualsiasi tentativo di modifica produce un errore immediato nel punto in cui viene effettuato. Raccoglie tutto ciò che si conosce del target prima dell'esecuzione:
 - la specifica OpenAPI dereferenziata come mappa strutturata di endpoint, parametri e schemi;
 - i parametri di tuning specifici per ogni test ricavati dalla configurazione;
 - l'adapter del gateway iniettato prima dell'avvio della pipeline;
 - le credenziali, risolte dal loader a partire dal file `.env` e congelate nel **TargetContext** tramite la configurazione `frozen`.⁴
- **TestContext** - mutabile, raccoglie ciò che la pipeline produce. L'accesso è strutturato in tre canali tipizzati con responsabilità distinte, anziché in un dizionario ad accesso libero:
 - canale dei token: organizza le credenziali per ruolo, con accesso tramite un identificatore definito dal contratto del sistema, non tramite chiavi stringa arbitrarie;
 - canale di teardown: mantiene un registro Last In, First Out (LIFO) delle risorse create durante la run, garantendo che il cleanup segua l'ordine inverso della creazione, come discusso nella Sezione 4.3.8;
 - canale dei dati condivisi: consente a test in batch successivi di consumare risultati prodotti da batch precedenti, senza che tra i moduli vengano introdotte dipendenze di importazione.

L'implementazione concreta di questa separazione, inclusi i tipi e i meccanismi di accesso, è descritta nella Sezione 4.1.3.

Qualsiasi informazione scoperta durante l'esecuzione (endpoint presenti sul target ma non dichiarati nella specifica, token acquisiti, risorse create) appartiene per definizione allo

⁴L'accesso avviene esclusivamente tramite i campi tipizzati del modello Pydantic costruito a runtime: non esiste un dizionario libero né accesso diretto ai valori grezzi.

stato dell'assessment e non alla conoscenza preliminare del target: viene quindi registrata nel contesto di esecuzione, non in quello del target immutabile. La separazione fa sì che la base informativa da cui ogni verdetto dipende non possa essere alterata durante la run, condizione necessaria perché la riproducibilità descritta nella Sezione 3.1.4 sia dimostrabile empiricamente e non solo enunciabile come proprietà attesa.

3.2.4 Gateway-Agnostic Adapter (D1.P6)

La Sezione 3.1.1 stabilisce che la conoscenza del target applicativo deriva esclusivamente dalla specifica OpenAPI e dal `config.yaml`. La proprietà presentata in questa sezione estende lo stesso principio di agnosticismo a un livello distinto, lo strato di controllo del gateway: alcune garanzie di sicurezza, come la verifica che i timeout sui servizi upstream siano configurati, che il plugin di circuit breaking sia attivo o che le credenziali di servizio non siano hardcoded nelle variabili d'ambiente, non sono osservabili interrogando gli endpoint esposti ai client. Queste informazioni risiedono nel piano di configurazione amministrativo del gateway e richiedono un canale di accesso separato dai normali probe HTTP.

Il pattern scelto è un'interfaccia astratta di sola lettura verso il gateway: i test `WHITE_BOX` vi accedono esclusivamente tramite i metodi che questa interfaccia espone, senza contenere nessun riferimento al gateway specifico installato nel deployment. L'implementazione concreta viene selezionata a runtime in base alla configurazione e iniettata nel contesto del target prima che la pipeline abbia inizio. Se la Admin API del gateway non è configurata o non è raggiungibile, tutti i test `WHITE_BOX` transitano a `SKIP` con motivazione esplicita, senza errori a cascata. I dettagli dell'interfaccia e dell'implementazione Kong sono descritti nella Sezione 4.5.1.

Il vantaggio è che aggiungere supporto per un gateway diverso richiede di implementare una nuova classe concreta che soddisfi l'interfaccia, senza toccare nessuno dei test che la utilizzano: la logica di verifica non cambia, cambia solo il meccanismo con cui i dati di configurazione vengono recuperati. Il costo è che l'interfaccia può esprimere solo le verifiche comuni a tutti i gateway supportati: controlli che richiedano funzionalità vendor-specific non generalizzabili rimangono fuori dal perimetro dell'astrazione, tensione analizzata nel dettaglio nella Sezione 6.1.2.

3.2.5 Streaming Evidence Store (D4.P1)

Un finding prodotto da un test acquista valore dimostrabile solo se accompagnato dall'evidenza HTTP che lo supporta: la request inviata, la response ricevuta, il timestamp e gli identificatori di traccia. Senza di essa, l'esito del test è un'asserzione non verificabile da terzi, priva del requisito di non-repudiabilità che un audit trail tecnico richiede.

Il design originale dell'Evidence Store utilizzava un buffer a capacità fissa in memoria: i record più vecchi venivano rimossi silenziosamente quando il buffer si riempiva, generando riferimenti a record inesistenti nell'audit trail. Il risultato era un trail rotto senza che il sistema producesse nessun errore visibile. Il difetto era strutturale, non correggibile aumentando la capacità: qualsiasi limite fisso avrebbe trasformato la capacità in un parametro di tuning il cui valore corretto dipende da ciò che si vuole misurare, informazione non disponibile a priori.

L'Evidence Store scrive ogni record su file locali con flush immediato, senza buffering in memoria e senza dipendenze verso database o storage remoto: ogni record è permanente nel momento in cui viene prodotto e non esiste un limite superiore al numero di record per test. Il ciclo di vita dettagliato del componente è descritto nella Sezione 4.6.1. La scelta di operare esclusivamente su file locali è anche la condizione che abilita quanto descritto nella sottosezione seguente.

3.2.6 Assenza di Dipendenze di Stato Esterno (D3.P5)

APIGuard Assurance non ha dipendenze di stato esterno a runtime: tutto lo stato mutabile del processo risiede in memoria o in file locali, e le uniche connessioni di rete aperte durante un run sono quelle verso il target applicativo e, opzionalmente, verso la Admin API del gateway. Entrambe sono connessioni di test verso il sistema da valutare, non dipendenze infrastrutturali del tool. Un tool che richiede database, message broker o cache esterna trasferisce la propria affidabilità a servizi esterni: l'exit code al termine dell'esecuzione riflette lo stato dell'assessment solo se tutti quei servizi erano disponibili e corretti, una garanzia che il tool non può fornire autonomamente.

Nessun client di database, cache distribuita o Software Development Kit (SDK) di storage remoto figura tra le dipendenze obbligatorie del progetto: il deployment si riduce all'installazione del package e alla fornitura del file di configurazione. Questa assenza è anche la condizione che rende il canale di comunicazione con la pipeline, descritto nella Sezione 6.2.1, strutturalmente affidabile: l'exit code al termine dell'esecuzione riflette esclusivamente lo stato dell'assessment, non quello di servizi accessori che potrebbero fallire in modo indipendente.

3.3 Domini di Sicurezza e Box Gradient Applicato

Le sezioni precedenti hanno definito le proprietà architetture e strutturali su cui il sistema si fonda. Questa sezione definisce cosa il sistema misura e secondo quale logica organizza le proprie misure: è il punto di convergenza tra la tassonomia delle minacce discussa nel Capitolo 2, dove le categorie di vulnerabilità sono state analizzate in quanto

problema, e la struttura operativa del tool, dove quelle stesse categorie diventano il perimetro di garanzie che un assessment sistematico è tenuto a coprire.

3.3.1 Tassonomia degli Otto Domini di Assurance

La superficie di sicurezza di un'API esposta tramite gateway non è omogenea. Alcune garanzie sono verificabili senza nessuna conoscenza del sistema, interrogando semplicemente gli endpoint esposti senza credenziali; altre richiedono uno stato autenticato; altre ancora diventano accessibili solo attraverso il piano di configurazione amministrativo del gateway. La tassonomia adottata riconosce questa struttura di precondizioni, organizzando le garanzie in domini che rispecchiano categorie semantiche distinte di rischio.

La Tabella 3.2 li riporta con il loro titolo e l'ambito di verifica che ciascuno racchiude.

Dominio	Titolo	Ambito di verifica
D0	API Discovery	Inventario degli endpoint esposti; rilevazione di Shadow API non documentate nella specifica.
D1	Identity & Authentication	Meccanismi di riconoscimento del chiamante; robustezza del trasporto delle credenziali; ciclo di vita e revoca dei JWT.
D2	Authorization	Logiche di controllo degli accessi; difetti di segregazione orizzontale e verticale (Role-Based Access Control (RBAC), BOLA).
D3	Data Integrity	Coerenza strutturale dei payload; implementazione delle firme crittografiche (Hash-based Message Authentication Code (HMAC)).
D4	Availability & Resilience	Difese contro l'esaurimento delle risorse; audit di circuit breaker e soglie di rate limiting.
D5	Observability	Qualità e tempestività della telemetria generata dal sistema. (<i>Milestone 2</i>)
D6	Configuration & Hardening	Rafforzamento della configurazione del gateway; policy di routing; header di sicurezza HTTP.
D7	Business Logic & Sensitive Flows	Falle semantiche complesse: elusione dei vincoli di processo, SSRF, race condition su operazioni critiche.

Tabella 3.2: Domini di Assurance di APIGuard Assurance.

La sequenza dei primi tre domini rispecchia una catena logica di precondizioni: non ha senso verificare l'autorizzazione (D2) senza avere prima verificato che l'autenticazione funzioni (D1), così come non ha senso verificare l'autenticazione senza conoscere la superficie di attacco (D0). Un assessment che verifichi D2 in assenza di certezze su

D1 raccoglie evidenza su un sistema il cui comportamento in caso di autenticazione compromessa è ignoto. I domini D3-D7 non seguono la stessa dipendenza lineare: appartengono a famiglie semantiche distinte, ciascuna con le proprie precondizioni di accesso, senza vincoli di ordinamento reciproco tra di loro. Questa catena logica si riflette nelle dipendenze dichiarate dai singoli test che operano su questi domini, il cui meccanismo di risoluzione è descritto nella Sezione 3.2.2.

Domain-5 occupa una posizione particolare nella tassonomia: il dominio è architetturealmente predisposto nella struttura del sistema, ma i test corrispondenti sono pianificati per la Milestone 2. La numerazione è riservata intenzionalmente per preservare la coerenza della sequenza nelle versioni successive.

3.3.2 Il Box Gradient come Mapping dalle Precondizioni al Privilegio (D3.P3)

La classificazione Black/Grey/White Box del testing, introdotta nella Sezione 2.4.2 come asimmetria informativa tra tester e sistema, viene derivata in APIGuard Assurance dalle precondizioni concrete che ogni garanzia impone al tester: non è una categoria assegnata retroattivamente ai test, ma il prodotto di ciò che ciascuna verifica richiede prima ancora di poter iniziare. Alcune garanzie sono fisicamente inaccessibili senza uno specifico livello di privilegio, e quella inaccessibilità è il criterio che determina il livello.

Il mapping adottato, sintetizzato nella Tabella 3.3, correla la strategia operativa alle precondizioni concrete che ogni classe di garanzie impone al tester. Il livello di visibilità di ogni test è determinato da ciò che la verifica richiede per essere condotta correttamente.

Strategia	Prerequisiti	Razionale
BLACK_BOX	Nessuno (solo accesso di rete)	Controlli perimetrali applicati dal gateway a qualsiasi interlocutore; simulazione di un attaccante esterno senza credenziali.
GREY_BOX	Token per ≥ 2 ruoli distinti	Garanzie che presuppongono stato autenticato con identità di ruolo distinte; inaccessibili senza credenziali valide.
WHITE_BOX	Accesso Admin API del gateway	Configurazione statica verificabile per ispezione diretta tramite piano di controllo del gateway.

Tabella 3.3: Box Gradient applicato in APIGuard Assurance.

I controlli perimetrali applicati dal gateway a qualsiasi interlocutore, autenticato o meno, non presuppongono nessuna conoscenza dell'infrastruttura interna: il risultato atteso è lo

stesso indipendentemente da chi effettua la richiesta. Eseguirli senza credenziali è l'unico modo per ottenere evidenza non contaminata da privilegi impliciti, ed è per questo che i test `BLACK_BOX` sono la scelta naturale per questa classe di garanzie.

Un controllo di autorizzazione RBAC, al contrario, richiede che il sistema si trovi in uno stato autenticato con almeno due identità di ruolo distinto. Senza quello stato il test non raggiunge nemmeno la logica che intende sondare: il gateway respinge la richiesta al livello di autenticazione, prima ancora che la logica di autorizzazione venga valutata, e il risultato osservato è indistinguibile da un corretto rifiuto di autenticazione. La strategia `GREY_BOX` è la risposta a questa precondizione.

I test `WHITE_BOX` occupano una posizione metodologicamente distinta. Alcune garanzie sono più efficacemente verificabili tramite ispezione diretta della configurazione del gateway che tramite probe empirici verso gli endpoint: la versione TLS supportata, la presenza e la parametrizzazione dei plugin di sicurezza, l'assenza di credenziali hardcoded nelle variabili d'ambiente. Un test empirico che cerchi di inferire questi parametri dall'esterno richiederebbe attacchi elaborati con esito incerto; un audit della configurazione li verifica in modo deterministico e riproducibile. È per questo che i test `WHITE_BOX` delegano la lettura della configurazione all'adapter del gateway descritto nella Sezione 3.2.4, anziché effettuare probe sull'interfaccia pubblica.

In contesti industriali, la disponibilità delle credenziali e dell'accesso amministrativo varia da deployment a deployment. Un'organizzazione che non dispone di accesso amministrativo al gateway può eseguire l'intera suite `BLACK_BOX` e `GREY_BOX`, mentre i test `WHITE_BOX` transitano a `SKIP` con motivazione esplicita. La Sezione 6.2.3 descrive come questa variabilità si traduca in scenari concreti di deployment industriale.

Capitolo 4

Implementazione

Le proprietà architetturali formalizzate nel Capitolo 3 si manifestano in questa sezione nella loro implementazione concreta. Il grado di trattamento rispecchia il peso strutturale di ciascuna: alcune definiscono l'organizzazione di interi componenti e sono trattate in sezioni dedicate; altre derivano dalle proprietà stabilite nel Capitolo 3 e vengono richiamate nel contesto del componente in cui si realizzano; altre ancora, di natura prevalentemente operativa, trovano collocazione nelle sezioni di questo capitolo pertinenti al loro locus, o nei capitoli successivi dedicati alla validazione e all'integrazione con i processi di sviluppo. La Tabella 3.1 costituisce la mappa di navigazione verso le sezioni dove ciascuna proprietà è introdotta.

Il Capitolo 3 ha percorso il primo livello di questo approccio top-down: ha stabilito i principi fondamentali del sistema, la metodologia di assessment, i domini di sicurezza e le proprietà architetturali che ne definiscono la struttura. Questo capitolo ne rappresenta il livello successivo nella discesa: i principi prendono forma in moduli Python, le proprietà si concretizzano in classi, meccanismi e scelte implementative specifiche. Dove un componente è stato introdotto nella sua motivazione architetturale, le sezioni seguenti aggiungono il dettaglio implementativo con un rimando esplicito alla sezione di origine.

4.1 La Pipeline di Esecuzione a Sette Fasi e il Core Engine

L'intero assessment si svolge in sette fasi sequenziali, orchestrate da `engine.py`. La sequenza rispecchia una dipendenza logica reale tra le operazioni, dove ogni fase produce le precondizioni che la successiva presuppone; tale dipendenza determina anche la semantica degli errori. Le fasi 1, 2 e 4 producono la configurazione validata, la mappa degli endpoint e il grafo delle dipendenze tra test: senza questi tre elementi nessun test può essere eseguito in modo significativo, e un errore in una qualsiasi di esse rende l'intero assessment privo di fondamento. Le fasi 5, 6 e 7 operano invece su un contesto già

validato, dove il fallimento è sempre recuperabile localmente: un test fallito produce un `TestResult(ERROR)` e la pipeline prosegue; un teardown parziale viene registrato come `WARNING` senza invalidare i risultati che lo precedono.

Fase	Cartella	File
1. Inizializzazione	<code>config/</code>	<code>loader.py</code>
2. OpenAPI Discovery	<code>discovery/</code>	<code>openapi.py</code>
3. Costruzione Contesti	<code>src/</code>	<code>engine.py</code>
4. Test Discovery e DAG	<code>tests/</code> , <code>core/</code>	<code>registry.py</code> , <code>dag.py</code>
5. Esecuzione	<code>src/</code>	<code>engine.py</code>
6. Teardown	<code>core/</code>	<code>context.py</code>
7. Report	<code>report/</code>	<code>renderer.py</code>

Tabella 4.1: Fasi della pipeline di assessment (1-2-4 bloccanti; 5-6-7 non-bloccanti).

La Tabella 4.1 riporta il mapping tra fase e modulo responsabile: ogni fase corrisponde a un modulo separato nel filesystem (Figura 4.1), e `engine.py` è il solo file autorizzato a importare da tutti gli strati (Sezione 3.2.1). Il dettaglio di ogni fase viene presentato nell'elenco sottostante; si nota che le eccezioni citate per le fasi bloccanti appartengono alla gerarchia custom analizzata nella Sezione 4.1.2, dove il loro phase mapping e le relative politiche di recovery sono descritte.

1. **Fase 1: Inizializzazione** (Sezione 3.1.3). `config/loader.py` legge e valida `config.yaml` contro lo schema Pydantic, interpola i segreti da variabili d'ambiente e costruisce il modello `RuntimeConfig`.
 - *In caso di errore:* `ConfigurationError` interrompe lo startup.
 - *Perché bloccante:* senza configurazione valida non esistono credenziali, URL di target né parametri di esecuzione; qualsiasi fase successiva opererebbe su basi non definite.
2. **Fase 2: OpenAPI Discovery** (Sezione 3.1.2). `discovery/openapi.py` scarica la specifica OpenAPI del target, dereferenzia i `$ref`¹ tramite `prance`, la valida con `openapi-spec-validator` e costruisce l'`AttackSurface`: la mappa strutturata degli endpoint che ogni test interroga per determinare il proprio scope.
 - *In caso di errore:* `OpenAPILoadError` interrompe lo startup.

¹Le specifiche OpenAPI usano `$ref` per referenziare schemi riutilizzabili definiti altrove nel documento o in file esterni. La dereferenziazione sostituisce ogni `$ref` con il contenuto del componente referenziato, producendo una specifica autonoma e navigabile senza dipendenze esterne.

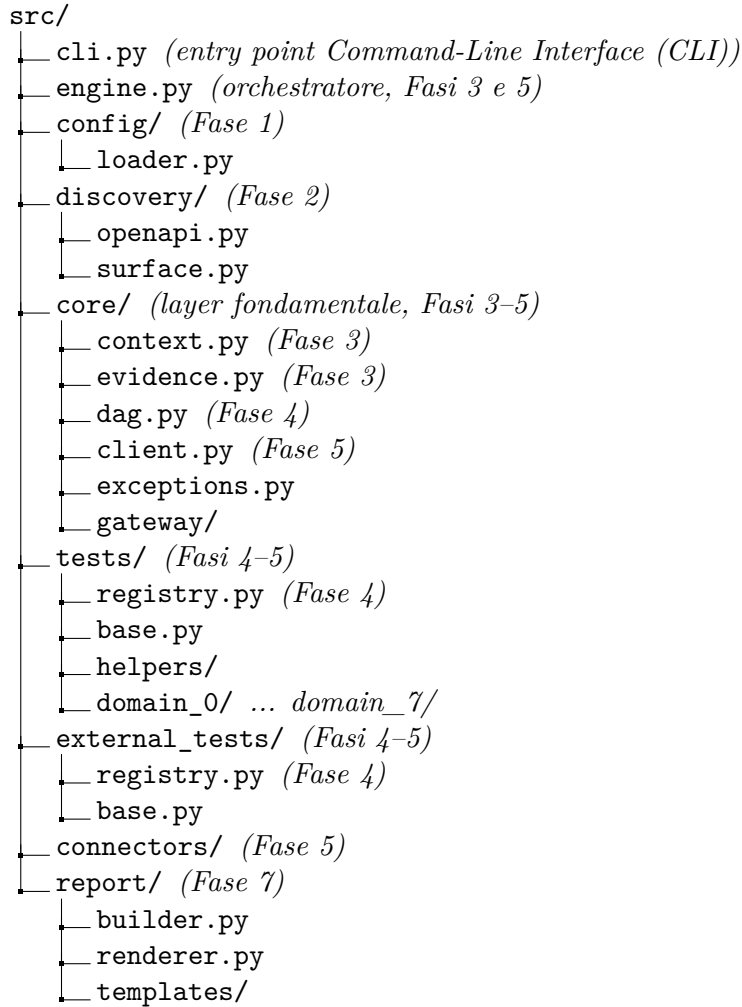


Figura 4.1: Layout di `src/` con mapping alle fasi della pipeline.

- *Perché bloccante:* la specifica OpenAPI è la sorgente esclusiva della conoscenza del target (Sezione 3.1.1); senza di essa nessun test può determinare su quali endpoint operare.

3. Fase 3: Costruzione Contesti. `engine.py` istanzia i tre oggetti di runtime condivisi: `TargetContext` (conoscenza del target frozen) e `TestContext` (stato dell'assessment mutabile), descritti nella Sezione 4.1.3, e `EvidenceStore` (streaming su disco), descritto nella Sezione 4.6.1.

- *In caso di errore:* nessuna eccezione prevista; la costruzione dei contesti non produce fallimenti recuperabili.

4. **Fase 4: Test Discovery e DAG.** `tests/registry.py` scopre dinamicamente i test disponibili via `pkgutil` e applica i filtri di proprietà e strategia configurati (Sezione 4.2.1); la lista risultante viene passata a `core/dag.py`, che costruisce il grafo delle dipendenze e calcola l'ordinamento topologico in batch (Sezione 4.2.2).
 - *In caso di errore:* `DAGCycleError` interrompe lo startup.
 - *Perché bloccante:* un ciclo nelle dipendenze rende il grafo non risolvibile; tentare l'esecuzione produrrebbe stallo o ordinamento non deterministico.
5. **Fase 5: Esecuzione.** `engine.py` itera sui `ScheduledBatch` prodotti dalla fase precedente ed esegue i test in ordine topologico, invocando `BaseTest.execute()` per i test nativi (Sezione 4.3) e `ExternalToolTest.execute()` per i test con tool esterni (Sezione 4.4); il meccanismo di isolamento degli errori è descritto nella Sezione 4.1.1.
 - *In caso di errore:* le eccezioni interne a ogni test sono catturate dal contratto di `execute()` e convertite in `TestResult(ERROR)`; la pipeline prosegue con il test successivo.
 - *Perché non-bloccante:* il fallimento di un singolo test è un dato dell'assessment, non un motivo per invalidare i risultati degli altri.
6. **Fase 6: Teardown.** `core/context.py` drena il registro delle risorse create durante l'esecuzione e tenta la loro rimozione in ordine LIFO tramite `SecurityClient`; il meccanismo è descritto nella Sezione 4.3.8.
 - *In caso di errore:* `TeardownError` registrato come `WARNING` strutturato; non invalida i risultati dell'assessment.
 - *Perché non-bloccante:* il mancato cleanup è un avvertimento operativo, non un'invalidazione scientifica dei finding già prodotti.
7. **Fase 7: Report.** `report/builder.py` aggrega i `TestResult` in un `ReportData` e produce tre output: `evidence.json` (archivio forense), `assessment_report.html` (report operativo con log completo delle transazioni) e `apiguard_report.json` (serializzazione strutturata per CI/CD), descritti nella Sezione 4.6.3.
 - *In caso di errore:* ogni sotto-operazione di 7 dispone di un proprio blocco di gestione indipendente; un fallimento su un output non impedisce la produzione degli altri.

Il ruolo di `engine.py` è limitato all'orchestrazione pura: esegue le fasi nell'ordine corretto, trasferisce i contesti da una fase all'altra, e non contiene logica di dominio di nessun tipo (né regole di sicurezza, né criteri di valutazione, né conoscenza del formato della specifica OpenAPI). La scelta ha una ragione strutturale: un orchestratore che incorpora logica di dominio diventa il punto di concentrazione delle dipendenze, rendendo difficile testare i componenti in isolamento e trasformando ogni sostituzione di fase in un'operazione ad alto rischio di regressione. La coerenza con il principio del flusso monodirezionale (Sezione 3.2.1) è verificabile per ispezione diretta: `engine.py` importa da `core/`, `discovery/`, `tests/` e `report/`, ma nessuno di questi moduli importa da `engine.py`.

4.1.1 Fail-Safe Error Isolation (D4.P3)

Il contratto tra l'orchestratore (`engine.py`) e i test individuali si caratterizza per un'assenza strutturale: nessun blocco `try/except` racchiude la chiamata a `test.execute()` nell'engine. Si tratta di una scelta deliberata, che delega la responsabilità della gestione delle eccezioni a livello di test, impedendo che una singola probe possa interrompere l'esecuzione delle altre per errori non previsti. Il perimetro di responsabilità dell'engine termina alla firma del contratto: invoca `test.execute()` e riceve un `TestResult`; qualsiasi condizione interna al test, normale o anomala, non attraversa quel confine.

Perché questo contratto funzioni, `BaseTest.execute()` deve garantire due invarianti: restituire sempre un `TestResult` e non propagare mai eccezioni verso l'esterno. La realizzazione concreta è un blocco `try/except Exception` all'interno di `execute()`: in caso di eccezione imprevista, l'helper `_make_error()` costruisce un `TestResult(status=ERROR, message=str(e))` con il traceback troncato a 500 caratteri. Un test con un bug interno, una risposta HTTP inattesa o un timeout non gestito produce un `TestResult(ERROR)` registrato nell'evidenza con il proprio traceback; la pipeline prosegue con il test successivo nel batch senza interruzioni. La solidità complessiva del sistema non è condizionata dalla correttezza di nessun singolo test.

4.1.2 Gerarchia delle Eccezioni con Phase Mapping (D4.P4)

Il meccanismo che rende possibile la distinzione tra errori bloccanti e non-bloccanti descritto in Tabella 4.1 è la gerarchia delle eccezioni custom. Tutte le eccezioni del tool ereditano da `ToolBaseError`, radice comune della gerarchia. La struttura offre al codice chiamante due granularità di intercettazione: un `except ToolBaseError` copre qualsiasi condizione di errore del tool in un unico punto; scendere su un'eccezione specifica, come `except ConfigurationError` o `except DAGCycleError`, permette di distinguere la causa e rispondere in modo differenziato. Tale granularità rispecchia in modo diretto la distinzione tra errori bloccanti e non-bloccanti stabilita a livello

architetturale, concretizzandosi in 11 classi, distribuite su 3 file, riportate nella Tabella 4.2 con la relativa fase.

Classe	File	Comportamento
<code>ToolBaseError</code>	<code>exceptions.py</code>	Radice della gerarchia; non si solleva direttamente
<code>ConfigurationError</code>	<code>exceptions.py</code>	Fase 1: fatale, config non valida; blocca lo startup
<code>OpenAPILoadError</code>	<code>exceptions.py</code>	Fase 2: fatale, spec irraggiungibile o invalida; blocca lo startup
<code>DAGCycleError</code>	<code>exceptions.py</code>	Fase 4: fatale, ciclo nelle dipendenze; blocca lo startup
<code>AuthenticationSetupError</code>	<code>exceptions.py</code>	Helper auth: fatale se le credenziali configurate sono invalide
<code>SecurityClientError</code>	<code>exceptions.py</code>	Fase 5: recuperata, produce <code>TestResult(ERROR)</code>
<code>ExternalToolError</code>	<code>exceptions.py</code>	Fase 5: recuperata, produce <code>TestResult(ERROR)</code>
<code>TeardownError</code>	<code>exceptions.py</code>	Fase 6: recuperata, registrata come <code>WARNING</code>
<code>GatewayAdapterError</code>	<code>gateway/base.py</code>	Fase 5: recuperata, produce <code>TestResult(ERROR)</code>
<code>SeedGeneratorFetchError</code>	<code>seed_generator.py</code>	CLI <code>generate-seed</code> : catturata in <code>cli.py</code> , exit code 10
<code>SeedGeneratorParseError</code>	<code>seed_generator.py</code>	CLI <code>generate-seed</code> : catturata in <code>cli.py</code> , exit code 10

Tabella 4.2: Gerarchia delle eccezioni custom (Fasi 1–4 bloccanti; Fasi 5–6 recuperate localmente).

La prima scelta che riflette una decisione di design consapevole e non un vincolo tecnico, riguarda la collocazione di `GatewayAdapterError` in `gateway/base.py` invece che in `exceptions.py`. La scelta segue il principio di locality: l’eccezione è definita nello stesso file dell’interfaccia che la solleva, rendendo il contratto dell’ABC autosufficiente. Un consumer di `BaseGatewayAdapter` trova tutto ciò che gli serve in un unico modulo, senza dover importare da `core/exceptions.py`.

A questa si affianca una seconda scelta altrettanto deliberata: l’assenza di `ExternalToolNotFoundError`. Un tool esterno mancante non è una condizione di errore imprevista: è una condizione operativa attesa, gestita a monte durante la Phase 4 tramite `is_available()` nel registry dei connector, come descritto nella Sezione 4.4.2. Il test che dipende da un tool assente riceve uno stato `SKIP` con motivo esplicito, non un’eccezione. Introdurre

`ExternalToolNotFoundError` avrebbe aggiunto una classe senza aggiungere semantica: la condizione è già rappresentata da `TestResult(status=SKIP)`.

4.1.3 Costruzione dei Contesti di Esecuzione (D1.P3)

In Phase 3, l'engine assembla i due oggetti che attraversano l'intera pipeline: `TargetContext` e `TestContext`. La motivazione architetturale della loro separazione è discussa nella Sezione 3.2.3.

`TargetContext` è dichiarato `frozen=True` a livello di modello Pydantic: qualsiasi tentativo di scrittura dopo la costruzione produce un errore immediato nel punto in cui viene effettuato. Raccoglie tutto ciò che il sistema conosce del target prima che la pipeline inizi: l'`AttackSurface` costruita dalla specifica OpenAPI in Phase 2, le credenziali risolte dal loader, i parametri di tuning per singolo test, la configurazione dei tool esterni e l'adapter del gateway iniettato se configurato. Ogni test che legge `TargetContext` ha la garanzia assoluta che i dati che legge sono identici a quelli letti da ogni altro test nella stessa run.

`TestContext` è mutabile, ma l'accesso non avviene tramite un dizionario libero: lo stato è organizzato in tre canali tipizzati con responsabilità distinte. Il canale token gestisce le credenziali acquisite durante la run tramite `set_token/get_token/has_token`, con chiavi che sono costanti nominate (`ROLE_ADMIN`, `ROLE_USER_A`, `ROLE_USER_B`) e non stringhe arbitrarie. Il canale teardown mantiene il registro LIFO delle risorse create, descritto nella Sezione 4.3.8. Il canale dati condivisi permette a test in batch successivi di consumare risultati prodotti da batch precedenti tramite `set_shared/get_shared`, con la convenzione `"{test_id}.{nome_dato}"` che rende esplicito quale test ha prodotto ogni valore.

4.2 Discovery Dinamico e Risoluzione del Grafo

Come stabilito nella Sezione 3.2.2 e anticipato nella Fase 4 dell'elenco nella Sezione 4.1, il DAG delle dipendenze tra test non è risolto dall'engine direttamente, ma delegato a due componenti con responsabilità distinte: il `TestRegistry`, che determina a runtime quali test esistono per ispezione del filesystem, e il `DAGScheduler`, che riceve la lista dei test scoperti e calcola l'ordine topologico a partire dalle dipendenze dichiarate. Il registry opera su moduli e classi Python; lo scheduler opera su grafi di stringhe. La separazione riflette la differenza di dominio tra le due operazioni: una riguarda la composizione del set di test attivi, l'altra la loro sequenza di esecuzione.

4.2.1 Discovery Dinamico via pkgutil (D2.P1)

Il `TestRegistry` in `src/tests/registry.py` scopre le classi concrete di test a runtime attraverso tre fasi sequenziali, denominate R1, R2 e R3, senza fare ricorso ad alcun elenco di registrazione manuale:

1. **Phase R1: Scansione del package.** La scansione del package `src.tests` è affidata a `pkgutil.walk_packages()`, che ne attraversa l'intera struttura importando tutti i moduli il cui nome finale rispetta il prefisso `test_`. Il meccanismo è indipendente dall'organizzazione delle sottodirectory: qualsiasi directory di `src/tests/` configurata come package Python con il proprio `__init__.py` viene inclusa automaticamente, a prescindere dal nome. Per convenzione, i test sono organizzati in directory `domain_N/`, una per ciascun dominio di assurance, ma il registry non vincola questa struttura. Ogni file di test adotta la convenzione `test_N_M_description.py`, dove N è il numero di dominio, M il numero progressivo del test all'interno del dominio e `description` una label in `snake_case`; il `test_id` dichiarato nella `ClassVar` segue il formato puntato corrispondente N.M. Errori di importazione nei singoli moduli vengono catturati e registrati come `WARNING` strutturato senza interrompere la scansione degli altri.
2. **Phase R2: Estrazione delle classi concrete.** Dai moduli importati vengono estratte le sottoclassi concrete di `BaseTest` tramite `inspect.getmembers()`, scaricando la classe base stessa, le sottoclassi astratte con metodi non implementati, e le classi prive dei `ClassVar` obbligatori verificati da `has_required_metadata()`.
3. **Phase R3: Filtraggio per priorità e strategia.** Vengono trattenuti i test la cui `priority` non supera la soglia `min_priority` e la cui `strategy` è inclusa nell'insieme delle strategie abilitate in `config.yaml`. Il filtraggio avviene per classe, prima dell'istanziamento, e ogni test escluso viene registrato a livello `DEBUG` con il motivo dell'esclusione.

Il meccanismo ha una conseguenza diretta sull'estensibilità del sistema: aggiungere un test richiede la creazione di un file nella directory di dominio corretta rispettando la convenzione `test_N_M_description.py` e implementare il contratto di `BaseTest` con i `ClassVar` obbligatori; nessun altro file deve essere modificato, nessuna registrazione manuale è necessaria.

L'`ExternalTestRegistry` in `src/external_tests/registry.py` segue le stesse fasi R1-R3, adattate alla gerarchia `ExternalToolTest` e alla convenzione di nomenclatura `ext_test_*.py`, con una fase aggiuntiva denominata R4. Dopo il filtraggio, il registry raggruppa i test sopravvissuti per `tool_name` e chiama `is_available()` una sola volta per gruppo: se il tool è presente, istanzia un connector condiviso e lo inietta in tutti i test del gruppo tramite `_injected_connector`; se il tool è assente, imposta

`_skip_reason_from_registry` su ogni test del gruppo, in modo che `execute()` restituisca `SKIP` senza tentare l'invocazione. Un solo `WARNING` viene emesso per tool mancante, indipendentemente da quanti test ne dipendono.

Lo stesso principio di estensibilità vale per i test esterni: aggiungere un nuovo `ExternalToolTest` richiede la creazione di un file rispettando la nomenclatura `ext_test_*.py`, implementare il contratto di `ExternalToolTest` con i `ClassVar` obbligatori e dichiarare il `tool_name` corrispondente al connector. Questa proprietà costituisce il prerequisito tecnico del plugin system delineato come traiettoria futura nella Sezione 6.3: un package esterno che aggiunge la propria directory al `sys.path` e rispetta le convenzioni di nomenclatura viene rilevato automaticamente al successivo avvio della pipeline, senza modifiche al core.

4.2.2 Scheduling Topologico e DAGScheduler (D2.P2)

Il `DAGScheduler` in `src/core/dag.py` riceve la lista dei `test_id` prodotta dal registry (Sezione 4.2.1) e restituisce una sequenza ordinata di `ScheduledBatch`: ciascun batch raggruppa i test che possono essere eseguiti nello stesso livello topologico perché le loro dipendenze sono già state soddisfatte nei batch precedenti. Il meccanismo di risoluzione, introdotto a livello architetturale nella Sezione 3.2.2, si concretizza qui nel pattern `get_ready() / done()` di `graphlib.TopologicalSorter`, la libreria standard Python introdotta nella versione 3.9. Il `DAGScheduler` costruisce, a partire dal campo `depends_on` di ogni test, una mappa da ogni `test_id` all'insieme dei suoi predecessori diretti, e la passa a `TopologicalSorter` come input per la risoluzione del grafo.² Chiama poi `prepare()` per intercettare eventuali cicli prima dell'esecuzione, e drena l'ordinamento livello per livello: ogni chiamata a `get_ready()` restituisce i nodi pronti, `done(*ready_nodes)` sblocca quelli che da essi dipendono, e ogni livello produce uno `ScheduledBatch`.³

`ScheduledBatch` è un `dataclass(frozen=True)` con due campi: `batch_index` (intero che indica il livello topologico incrementando a partire da zero) e `test_ids` (lista ordinata di stringhe). L'immutabilità garantisce che l'engine non possa alterare un batch durante l'esecuzione. Il determinismo all'interno di ogni batch, requisito discusso nella Sezione 3.1.4, dipende da una singola riga: `ready_nodes = sorted(sorter.get_ready())`. `graphlib.TopologicalSorter.get_ready()` restituisce i nodi pronti senza garantire nessun ordine sull'insieme estratto: a parità di grafo e di dipendenze dichiarate, l'ordine di iterazione può variare tra run, versioni di Python o piattaforme. L'ordinamento lessicografico esplicito sul `test_id` è l'unica fonte di ordine deterministico all'interno

²Ogni test dichiara solo le dipendenze immediate: se il test A dipende da B e B dipende da C, è sufficiente che A dichiari `depends_on = ["B"]`. Il vincolo su C è già catturato dalla dipendenza di B su C, e `TopologicalSorter` lo propaga automaticamente nel calcolo dell'ordinamento.

³Passare un nodo a `done()` segnala al sorter che quel `test_id` è stato completato: tutti i test che lo dichiarano in `depends_on` vedono soddisfatto quel vincolo e diventano eleggibili per la successiva chiamata a `get_ready()`.

di un batch: due run con le stesse dipendenze dichiarate producono sempre la stessa sequenza di esecuzione, condizione necessaria perché D3.P4 sia verificabile empiricamente e non solo enunciabile.

Un caso limite, documentato nella Sezione 3.2.2 a livello architetturale, è gestito dalla stall detection: `is_active()` restituisce `True` ma `get_ready()` non produce nodi, segnalando che il sorter è in uno stato irrecuperabile nonostante l'assenza di cicli dichiarati. Il `DAGScheduler` calcola in questo caso l'insieme dei `test_id` mai estratti, li registra in un `ERROR` strutturato con il campo `unscheduled_test_ids`, e interrompe il drain. Il log fornisce all'operatore l'elenco esatto dei test coinvolti senza richiedere la lettura di un traceback generico.

4.3 Architettura dei Test Nativi e Contratto `BaseTest`

I test nativi sono le unità di verifica che comunicano direttamente con il target tramite `SecurityClient` e producono evidenza HTTP osservabile. Il loro ciclo di vita nella pipeline, dalla scoperta in Fase 4 all'invocazione in Fase 5, è descritto nella Sezione 4.1; questa sezione ne descrive la struttura interna: il contratto che ogni test deve rispettare per essere scoperto e orchestrato correttamente, le invarianti sul risultato che il sistema applica per costruzione, il meccanismo di autenticazione condivisa, le regole di tracciabilità normativa, la policy con cui le probe HTTP vengono interpretate, il meccanismo di risoluzione dei path parametrici e il catalogo dei payload di attacco, e il meccanismo di cleanup delle risorse create durante la verifica. La Milestone 1 implementa test per i domini D0, D1, D2, D3, D4, D6 e D7; il Domain-5 (Observability) è architetturalmente predisposto e pianificato per la Milestone 2.

4.3.1 Gerarchia Duale `BaseTest` / `ExternalToolTest` (D1.P4)

Il sistema ospita due categorie di test con contratti distinti: i test nativi estendono `BaseTest` in `src/tests/base.py`; i test con tool esterni estendono `ExternalToolTest` in `src/external_tests/base.py`. Le due classi base sono gerarchie parallele e indipendenti, per ragioni tecniche precise che la struttura del sistema rende evidenti.

La scelta di mantenere le due gerarchie separate risponde a un problema concreto di ereditarietà indesiderata. `BaseTest` espone metodi come `_log_transaction()` e `_probe_endpoint()`, che operano tramite `SecurityClient` per eseguire richieste HTTP contro il target applicativo. Un `ExternalToolTest` non usa `SecurityClient`: delega la raccolta di dati al proprio connector, che invoca un processo di sistema o una libreria Python. Fare ereditare `ExternalToolTest` da `BaseTest` avrebbe reso disponibile a ogni

test esterno un'interfaccia pensata per un pattern di integrazione diverso, ampliando la superficie di errore e oscurando il contratto che il test deve effettivamente rispettare.

Dal punto di vista dell'engine, la distinzione è quasi invisibile: entrambe le gerarchie producono un `TestResult`, vengono scoperte dal registry con gli stessi meccanismi di Phase R1-R3 descritti nella Sezione 4.2.1 (con l'aggiunta di Phase R4 per i test esterni), e vengono schedate dal `DAGScheduler`. L'unica differenza visibile in `engine.py` è un `isinstance` check al momento dell'invocazione, che seleziona la firma corretta di `execute()`. Il campo `TestResult.source` con tipo `Literal["native", "external"]` propaga la distinzione fino al report, dove i risultati nativi e quelli di tool specializzati vengono riportati con sezioni visivamente distinte all'interno dello stesso dominio.

Il contratto che `BaseTest` impone a ogni implementazione concreta si articola in due livelli:

- **Dichiarazione statica via `ClassVar`:** ogni test deve dichiarare `test_id` (identificatore univoco corrispondente all'ID nella metodologia), `domain` (dominio di assurance 0-7), `priority` (P0-P3), `strategy` (`BLACK_BOX`, `GREY_BOX`, `WHITE_BOX`), `depends_on` (lista di `test_id` predecessori per il DAG), `cwe_id` e `tags` (riferimenti normativi). `has_required_metadata()` scarta le classi con `ClassVar` incompleti prima che vengano istanziate, così le implementazioni parziali non raggiungono mai la pipeline.
- **Implementazione di `execute(target, ctx, store)`:** il metodo che garantisce le due invarianti discusse nella Sezione 4.1.1: restituire sempre un `TestResult` e non propagare mai eccezioni verso l'engine.

4.3.2 Invarianti di `TestResult` a Livello di Tipo (D4.P9)

Ogni test produce un `TestResult`, il modello Pydantic che trasporta lo stato finale della verifica verso il report. La coerenza tra lo stato di un `TestResult` e il suo contenuto, se affidata alla disciplina distribuita di implementazioni indipendenti, è una proprietà che il sistema non può garantire strutturalmente: uno stato `FAIL` senza evidenza, uno `SKIP` senza motivazione, un `PASS` con findings allegati sono tutti costruibili sintatticamente senza che il sistema se ne accorga, producendo entry di report formalmente validi ma privi di significato verificabile. La soluzione adottata sposta l'enforcement a livello di tipo, tramite un `@model_validator(mode="after")` in `src/core/models/results.py` che applica tre invarianti nel momento in cui ogni test costruisce il proprio risultato, ovvero alla chiamata di `_make_pass()`, `_make_fail()` o `_make_skip()`, prima che l'oggetto raggiunga qualsiasi consumer:

1. `status=FAIL` richiede `findings` non vuoto. Un FAIL senza evidenza non è un FAIL: è un'asserzione.
2. `status=SKIP` richiede `skip_reason` non None. Ogni SKIP deve essere documentabile e auditabile.
3. `status=PASS` richiede `findings` vuoto. Un PASS non può coesistere con violazioni dichiarate.

La centralizzazione in un singolo `model_validator` ha un effetto architetturale preciso: il contratto tra stato e contenuto del `TestResult` è enforced nel punto in cui l'oggetto viene costruito, indipendentemente da quale test lo costruisce e da quale helper utilizza. Una violazione solleva `pydantic.ValidationError` nel punto in cui l'helper `_make_pass()`, `_make_fail()` o `_make_skip()` costruisce l'oggetto, con un traceback che punta direttamente alla riga incriminata nel test responsabile. Ogni nuova implementazione di test ottiene la verifica automaticamente: il validator intercetta risultati inconsistenti nel punto stesso della costruzione, senza che il test debba contenere logica di validazione propria.

4.3.3 Auth Abstraction Layer e Idempotenza dei Token (D2.P5)

I test `GREY_BOX` non possono iniziare la propria probe senza token validi: il gateway respinge qualsiasi richiesta priva di credenziali prima che la logica di autorizzazione venga anche solo valutata. Con una parte significativa dei 18 test attivi in Milestone 1 in modalità `GREY_BOX`, distribuire la logica di login in ogni test produrrebbe autenticazioni ripetute verso lo stesso target per ruoli già acquisiti. `acquire_tokens()` in `src/tests/helpers/auth.py` risolve questo problema centralizzando la gestione dei token nel `TestContext`: alla chiamata, la funzione verifica prima se il `TestContext` contiene già un token valido per il ruolo richiesto tramite `ctx.has_token(role)`; solo se il token è assente esegue il login HTTP e lo scrive nel canale token tramite `ctx.set_token(role, token)`. Il numero di login effettivi verso il target è così esattamente uguale al numero di ruoli distinti configurati, indipendentemente da quanti test li richiedano.

Il dispatcher in `auth.py` legge il campo `auth_type` dal `config.yaml` e delega l'acquisizione dei token all'implementazione corrispondente, astruendo i test dal dettaglio del protocollo. Nella Milestone 1 sono supportati due metodi: `forgejo_token`, che acquisisce un token tramite l'endpoint Forgejo-specifico `POST /api/v1/users/{username}/tokens` con HTTP Basic Auth, e `jwt_login`, un flusso generico configurabile tramite `login_endpoint`, `username_body_field` e `token_response_path`, adatto a qualsiasi target che esponga un endpoint di login JSON, da crAPI a Django REST Framework. Aggiungere il supporto per un nuovo meccanismo richiede una nuova implementazione del flusso e un ramo aggiuntivo nel dispatcher; i test rimangono invariati perché il loro contratto con il layer di autenticazione è espresso esclusivamente in termini di ruolo richiesto.

4.3.4 Tracciabilità Normativa e Distinzione Finding/InfoNote (D5.P1, D5.P2)

Ogni test porta con sé, come `ClassVar`, i riferimenti normativi della garanzia che verifica: `cwe_id` per la classe di vulnerabilità nel CWE, e `tags` per le categorie OWASP, i riferimenti National Institute of Standards and Technology (NIST) e i documenti tecnici Request for Comments (RFC). Entrambi vengono propagati fino al modello `Finding` tramite il campo `references: list[str]` e riportati dal report HTML a fianco di ogni risultato: un analista può risalire direttamente dal finding al codice CWE, alla categoria OWASP API Security e alle sezioni normative pertinenti senza consultare documentazione esterna.

La tracciabilità presuppone una distinzione semantica precisa tra due tipi di annotazione che un test può produrre. Un `Finding` è evidenza di una violazione: appartiene esclusivamente ai risultati `FAIL`, il suo contenuto deve essere sufficiente a giustificare la valutazione negativa, e viene conteggiato nei totali aggregati del report. Un `InfoNote` è un'annotazione informativa allegabile a un risultato `PASS`: documenta contesto rilevante per l'analista senza alterare il verdetto né influenzare i contatori. La distinzione è architetturale: i contatori del report aggregano esclusivamente oggetti `Finding`, rendendo il totale affidabile e non influenzabile da osservazioni di contorno. La coerenza tra tipo di risultato e tipo di annotazione è enforced come descritto nella Sezione 4.3.2: il `model_validator` di `TestResult` garantisce che un `PASS` non possa coesistere con `Finding` nel momento stesso della costruzione dell'oggetto.

Il valore operativo della distinzione emerge con chiarezza nel test 4.3, che verifica la presenza di un meccanismo di circuit breaking sull'infrastruttura del gateway. Kong OSS, nella configurazione di Milestone 1, non implementa circuit breaking attivo ma espone healthcheck passivi che svolgono una funzione parzialmente equivalente. Il test registra `PASS` perché il requisito minimo è soddisfatto, e allega una `InfoNote` che documenta la differenza tra il compensating control osservato e il circuit breaking completo. L'analista ottiene un quadro preciso senza che il sistema produca né un falso `FAIL` né un `PASS` silenzioso che nasconde la sfumatura.

4.3.5 Oracle a Tre Esiti e Safe HTTP Probing Policy (D4.P7)

Ogni test nativo che interroga un endpoint del target riceve una risposta HTTP che il tool deve interpretare e tradurre in un verdetto preciso. Il problema non è banale: uno status `404 Not Found` su un endpoint parametrico come `/users/{id}/repos` può significare che il gateway ha rifiutato la richiesta per mancanza di autenticazione, che il path non esiste per il valore di `{id}` usato nel probe, o che l'endpoint è genuinamente non protetto ma il path era errato. Interpretare un `404` come `FAIL` in tutti i casi genera falsi positivi inaccettabili in un assessment che si propone come affidabile.

La policy adottata definisce un framework a tre esiti applicato a ogni probe HTTP nei test nativi: ogni risposta è classificata come **ENFORCED** (il controllo funziona), **BYPASSED** (il controllo è assente o mal configurato) o **INCONCLUSIVE** (il risultato non è interpretabile con certezza sufficiente per produrre un **Finding**). Il mapping specifico degli status code a ciascun esito dipende dalla natura del test: per i test **BLACK_BOX** che verificano l'enforcement dell'autenticazione, la forma è la seguente:

1. **ENFORCED**: la risposta è 401 o 403. Il gateway ha applicato un controllo di accesso sul chiamante non autenticato.
2. **BYPASSED**: la risposta è 2xx. Il gateway non ha richiesto autenticazione. Il controllo è assente o mal configurato.
3. **INCONCLUSIVE**: qualsiasi altro status. Il risultato non è interpretabile con certezza sufficiente per produrre un **Finding** ⁴.

Solo un esito **BYPASSED** produce un **Finding** e contribuisce a un risultato **FAIL**. Un esito **INCONCLUSIVE** viene registrato nell'evidenza con il suo status effettivo, ma non altera il verdetto del test.

La codifica a tre esiti presuppone che la risposta ricevuta rifletta il comportamento del controllo di accesso, non un effetto collaterale della probe stessa. Questa condizione regge per i path fissi della specifica, dove il test raggiunge una risorsa che esiste e la risposta è interpretabile. Per i path parametrici la situazione è più sottile: un endpoint come `/users/{id}/repos` ha una struttura valida, ma il valore concreto del parametro determina se quella specifica risorsa esiste sul target. Con un valore che non corrisponde a nessun oggetto reale, il gateway o il backend restituisce **404** perché la risorsa è assente, non perché stia applicando un controllo di accesso; il risultato osservato è indistinguibile da un corretto rifiuto di autenticazione. Classificare quel **404** come **INCONCLUSIVE** è l'unica risposta metodologicamente corretta. Da questa asimmetria nasce la segregazione in due tier: il Tier A raccoglie i path fissi e quelli parametrici per cui il seed è configurato (Sezione 4.3.6), dove la probe usa valori concreti verificati sul target e la risposta è interpretabile; il Tier B raccoglie i path parametrici senza seed, dove il risultato è **INCONCLUSIVE** e l'evidenza segnala all'operatore che configurare il seed risolverebbe l'ambiguità.

Un caso limite merita attenzione esplicita. Le probe con metodo **DELETE** su endpoint parametrici presentano un rischio operativo: se il gateway è mal configurato e non applica autenticazione, una richiesta **DELETE** con un path valido potrebbe eliminare

⁴Esempio osservato su Forgejo: senza path seed, il probe su `/api/v1/repos/{owner}/{repo}` usa il placeholder "1" per entrambi i parametri; Forgejo risponde **404** perché la risorsa non esiste, la richiesta non raggiunge il middleware di autenticazione e il risultato è **INCONCLUSIVE**. Con seed configurato, la stessa richiesta produce **ENFORCED** o **BYPASSED**.

risorse reali sul target. La policy prevede un fallback sicuro: quando il path seed non è configurato per un endpoint DELETE parametrico, il probe viene eseguito con il valore letterale "apiguard-probe" come sostituto del parametro. Un path con quel valore non corrisponde a nessuna risorsa reale su alcun sistema, garantendo che il probe non causi effetti collaterali indesiderati anche in presenza di un gateway non autenticato.

4.3.6 Path Seed System (D2.P7)

Il path seed system affronta una tensione strutturale dell'agnosticismo applicativo. Derivare tutta la conoscenza del target dalla specifica OpenAPI significa conoscere la topologia degli endpoint ma non i valori concreti dei loro parametri: `/users/{owner}/repos/{repo}` è un endpoint documentato, ma senza sapere quale `owner` e quale `repo` esistano realmente sul target, la probe usa un placeholder il cui path quasi certamente non esiste, producendo 404 e un risultato INCONCLUSIVE. Il seed risolve questa tensione fornendo per ogni parametro la possibilità di specificare un valore concreto verificato sul target⁵: la generazione tramite il comando CLI `generate-seed`, che interroga la specifica OpenAPI, produce un template da completare e inserire in `config.yaml`. Con il seed configurato per Forgejo, i path con `{owner}` e `{repo}` producono risposte interpretabili: ENFORCED se il gateway applica il controllo atteso, BYPASSED se non lo applica. La Sezione 5.3 riporta i dati quantitativi di questo effetto sull'assessment descritto in questo capitolo.

4.3.7 Catalogo dei Payload di Attacco (D2.P8)

I test nativi applicano il principio di separazione tra dati e logica di valutazione che, come verrà illustrato nella Sezione 4.4.1, governa anche il contratto dei connector: i dati di input alla verifica risiedono separati dalla logica che li interpreta. Nel caso dei test nativi, questo si concretizza nel catalogo dei payload. Le sequenze di URL usate dal test SSRF, gli header malformati per i test di input validation, e altri vettori di attacco risiedono in moduli puri in `src/tests/data/`, distinti dalla logica che li applica. Aggiungere un nuovo vettore richiede la modifica del catalogo, senza toccare il test che lo utilizza. La separazione ha una conseguenza operativa concreta in contesti enterprise, dove i payload di attacco sono soggetti a un processo di approvazione separato dal codice applicativo.

4.3.8 Teardown Best-Effort in Ordine LIFO (D4.P6)

I test che raggiungono la logica applicativa del target spesso creano risorse necessarie alla verifica: token di autenticazione temporanei, utenti di test, risorse effimere. Lasciare

⁵Un valore verificato è un identificatore di una risorsa realmente esistente nel deployment; nel caso di Forgejo: un utente, un repository, e così via.

queste risorse sul target al termine dell'assessment viola la proprietà di non-interferenza: un assessment successivo potrebbe incontrare uno stato alterato, e il target produttivo potrebbe accumulare risorse orfane nel tempo.

Il cleanup delle risorse si basa sulla registrazione esplicita al momento della creazione: ogni volta che un test crea una risorsa persistente, chiama immediatamente `ctx.register_resource_for_teardown(method, path, headers)` passando l'endpoint di cancellazione. La coppia creazione-registrazione avviene nella stessa transazione logica; se il test va in eccezione tra le due operazioni, la risorsa potrebbe non essere rimossa. Questa condizione è documentata come limite noto: il meccanismo di isolamento degli errori descritto nella Sezione 4.1.1 non può recuperare uno stato che non è mai stato registrato.

In Phase 6, l'engine chiama `ctx.drain_resources()`, che restituisce le risorse in ordine LIFO. Tale ordine inverso rispetto alla creazione è necessario per gestire correttamente le dipendenze tra risorse: se un test ha creato prima un repository e poi una issue al suo interno, la issue dipende dal repository; cancellarli nell'ordine di creazione eliminerebbe prima il repository, rendendo impossibile la cancellazione della issue. Invertendo l'ordine, la issue viene rimossa prima che il repository smetta di esistere. Per ogni risorsa restituita, l'engine tenta la chiamata HTTP di cancellazione tramite `SecurityClient`; un fallimento solleva `TeardownError`, catturato e registrato come `WARNING` strutturato. Il fallimento del teardown è un avvertimento operativo, non una condizione che invalida i risultati prodotti nelle fasi precedenti. La Sezione 5.3 riporta i dati della verifica empirica, che ha rilevato zero risorse residue su due run indipendenti.

4.4 Gerarchia dei Connettori e Integrazione Tool Esterni

La controparte della gerarchia duale introdotta nella Sezione 4.3.1 è `ExternalToolTest`: i test che delegano la raccolta di dati a tool specializzati invece di comunicare direttamente con il target tramite `SecurityClient`. Seguono lo stesso ciclo di vita nella pipeline (scoperti in Fase 4 dall'`ExternalTestRegistry` con l'aggiunta di Phase R4, invocati in Fase 5), ma con un pattern di integrazione strutturalmente diverso. Scanner di vulnerabilità, analizzatori TLS, fuzzer producono tipi di evidenza che un client HTTP generico non può raccogliere: la negoziazione TLS richiede dialogo a livello di protocollo, l'analisi con template nuclei richiede una base di signature aggiornata. Il layer dei connector è il punto in cui questa delega prende forma concreta: isola la logica di invocazione di ogni tool, normalizza il suo output in un modello comune e restituisce al test un oggetto strutturato invece del raw output del processo. In assenza di questo layer, ogni test esterno conterrebbe sia la logica di verifica della garanzia sia la meccanica di subprocess management o di importazione della libreria, accoppiando due responsabilità che evolvono per ragioni distinte.

4.4.1 Gerarchia a Tre Livelli e Separazione Dati/Valutazione (D1.P5, D2.P3)

La gerarchia dei connector si articola su tre livelli, ciascuno dei quali eredita solo i meccanismi pertinenti al proprio pattern di integrazione.

`BaseConnector` in `src/connectors/base.py` è il primo livello: un Abstract Base Class (ABC) puro con un solo `ClassVar` (`TOOL_NAME: str`) e tre metodi astratti che costituiscono il contratto universale verso il test. `is_available()` verifica se il tool è utilizzabile nell'ambiente corrente; `get_version()` restituisce la versione del tool per fini di audit; `run(target_url, timeout_seconds)` esegue il tool e restituisce un `ConnectorResult`. Il parametro `timeout_seconds` è privo di valore di default per design: ogni chiamante deve fornirlo esplicitamente dal `config.yaml`, rendendo impossibile dimenticarlo.

Il secondo livello si biforca a seconda della natura del tool:

- `BaseSubprocessConnector` serve i tool invocati come processo di sistema (`NucleiConnector`, `TestsslConnector`): fornisce implementazioni concrete di `is_available()` e `get_version()` basate su subprocess, e il metodo `_run_subprocess()` che gestisce esecuzione, timeout e parsing dell'output. La ricerca del binario avviene su tre canali, fermandosi alla primo soddisfatto, nel seguente ordine: binario locale nella directory `tools/` del progetto, poi `shutil.which()` sul PATH di sistema, infine una variabile d'ambiente per i deployment in cui il tool è esposto come microservizio HTTP. Il medesimo connector funziona così senza modifiche in sviluppo locale, in CI/CD e in ambienti containerizzati.
- `BaseLibraryConnector` serve i tool Python-native (`SslyzeConnector`): usa `importlib.util.find_spec()` per verificare la disponibilità della libreria senza invocare alcun subprocess, e legge la versione tramite `importlib.metadata`. La disponibilità è determinata dalla presenza del package nell'ambiente Python, non da un binario sul filesystem.

Le implementazioni concrete si limitano a dichiarare le `ClassVar` specifiche del tool⁶ e implementare `run()`, il cui contratto formalizza una separazione precisa tra raccolta di dati e valutazione della garanzia. Il connector restituisce un `ConnectorResult`, modello Pydantic frozen che trasporta `tool_name`, `tool_version`, `raw_output: dict[str, Any]`, `exit_code`, `execution_time_ms` e `timed_out`: dati grezzi, privi di qualsiasi giudizio sulla sicurezza del target.

La valutazione è responsabilità esclusiva dell'`ExternalToolTest`, che implementa `_evaluate()` e applica il proprio oracle al `ConnectorResult`. La ragione di questa separazione è la

⁶Per `BaseSubprocessConnector`: `TOOL_NAME`, `BINARY_NAME`, `LOCAL_TOOLS_SUBDIR` e `SERVICE_ENV_VAR`.
Per `BaseLibraryConnector`: `TOOL_NAME` e `LIBRARY_NAME`.

riusabilità: il `NucleiConnector` non sa nulla della garanzia che il test vuole verificare, e per questo può essere condiviso da test con logiche di valutazione completamente diverse. Un secondo test che vuole usare nuclei per injection scanning scriverebbe un `_evaluate()` diverso ma userebbe lo stesso connector, senza alcuna modifica alla logica di invocazione.

4.4.2 Lifecycle dei Connector e Dependency Injection (D2.P4)

Come descritto nella Sezione 4.2.1, la Phase R4 dell'`ExternalTestRegistry` raggruppa i test per `tool_name` e chiama `is_available()` una sola volta per gruppo. Il risultato di quella chiamata determina il lifecycle del connector per tutti i test del gruppo: disponibilità o assenza del tool si traduce in due comportamenti distinti, gestiti tramite due attributi di istanza di `ExternalToolTest`.

Se il tool è disponibile, il registry istanzia un connector condiviso e lo scrive in `_injected_connector` su ogni test del gruppo: tutti condividono la stessa istanza, costruita una volta sola. Se il tool è assente, il registry scrive invece in `_skip_reason_from_registry` la motivazione di skip già formulata. Al momento dell'esecuzione, `_run()` controlla questo campo come prima operazione: se valorizzato, restituisce `SKIP` immediatamente senza tentare la costruzione del connector né interrogare il filesystem. In entrambi i casi, la decisione è presa una volta sola in Phase R4 e propagata a tutti i test del gruppo tramite questi due attributi.

La conseguenza operativa misurabile è nel log di esecuzione. Con cinque test che dipendono da nuclei e il binario assente, il log produce un solo `WARNING` del tipo *"nuclei not found: 5 tests will SKIP"*, invece di cinque entry identiche. In un assessment con molti tool e molti test, questa differenza rende il log leggibile invece che ridondante. Il comportamento complessivo del sistema in presenza di infrastruttura parzialmente disponibile, compresi gli scenari con Admin API assente o master switch disattivato, è descritto nella Sezione 4.5.2.

4.4.3 Classificazione A/B dei Connector (D2.P6)

I connector sono classificati in due categorie che riflettono il loro rapporto con la copertura nativa. I connector di Categoria A coprono garanzie che i test nativi non possono raggiungere con soli probe HTTP: la loro assenza produce `SKIP` su verifiche altrimenti inaccessibili. I connector di Categoria B estendono la copertura su garanzie già verificate parzialmente dai test nativi, aggiungendo profondità senza essere necessari per l'assessment di base. Nella Milestone 1, `NucleiConnector` e `TestsslConnector/SslyzeConnector` appartengono entrambi alla Categoria A: il primo perché l'analisi di Shadow API tramite template specializzati non è replicabile con probe HTTP generici; i secondi perché la

negoziazione TLS richiede dialogo a livello di protocollo che supera le capacità di un client HTTP standard.

4.4.4 Isolamento della Dipendenza AGPL (D7.P2)

`sslyze`, la libreria Python per l'analisi TLS, è distribuita sotto licenza GNU Affero General Public License (AGPL) v3. Le dipendenze dichiarate in `[project.dependencies]` di `pyproject.toml` vengono installate automaticamente con qualsiasi `pip install apiguard-assurance`; includervi `sslyze` propagherebbe i suoi obblighi copyleft a chiunque distribuisca o utilizzi il tool, indipendentemente dall'uso effettivo della libreria. La sezione `[project.optional-dependencies]` del `pyproject.toml` è pensata esattamente per questo tipo di situazione: raccoglie le dipendenze con vincoli di licenza o peso computazionale che non devono essere imposti per default, lasciando all'operatore la scelta consapevole di installarle tramite un extra esplicito.

`sslyze` è dichiarata in `[project.optional-dependencies.sslyze]`. L'installazione esplicita tramite `pip install "apiguard-assurance[sslyze]"` abilita `ext.1.5.sslyze` e comporta l'accettazione consapevole della licenza AGPL per quella componente. In assenza dell'installazione, `SslyzeConnector.is_available()` restituisce `False` tramite `importlib.util.find_spec()`, il test riceve `SKIP` con motivazione esplicita, e il core rimane privo di qualsiasi import condizionale della libreria, isolandola strutturalmente. Il pattern è replicabile per qualsiasi dipendenza con vincoli analoghi configurando il `pyproject.toml` appositamente.

4.5 Implementazione del Gateway Adapter

La Sezione 3.2.4 ha stabilito il principio di agnosticismo verso il gateway e introdotto `BaseGatewayAdapter` come interfaccia astratta di sola lettura verso il piano di configurazione amministrativo. Questa sezione ne descrive la concretizzazione: l'implementazione Kong che viene iniettata nel `TargetContext` in Fase 3 e i tre livelli di degradazione controllata che il sistema applica quando parti dell'infrastruttura non sono disponibili.

4.5.1 KongGatewayAdapter: Accesso Read-Only all'Admin API

`KongGatewayAdapter` in `src/core/gateway/kong.py` è l'unica implementazione concreta di `BaseGatewayAdapter` nella Milestone 1, e realizza il principio dichiarato nella Sezione 3.2.4: i test `WHITE_BOX` 3.3, 4.2, 4.3 e 6.4 chiamano `target.gateway.get_plugins()`, `get_services()` e `get_upstreams()` tramite l'interfaccia astratta, senza contenere nessun riferimento a Kong. L'implementazione interroga la Admin API di Kong DB-less

tramite richieste HTTP GET. Una scelta implementativa merita particolare attenzione: il componente usa `httpx` direttamente invece di `SecurityClient`. Le chiamate alla Admin API sono audit di configurazione dell'infrastruttura, non traffico di test verso il target applicativo; appartengono a un confine di fiducia distinto, non devono apparire nell'`EvidenceStore` e non sono soggette alla policy di retry del `SecurityClient`. Un client `httpx` dedicato mantiene i due confini separati nel codice in modo esplicito e verificabile.

I metodi pubblici dell'adapter, `get_routes()`, `get_plugins()`, `get_services()`, `get_upstreams()` e `get_plugin_by_name()`, delegano tutti la lettura a `_fetch_paginated()`. La Admin API di Kong restituisce le collezioni in formato paginato: ogni risposta contiene un campo `"data"` con gli elementi della pagina corrente e un campo `"next"` con il path della pagina successiva, oppure `null` se non ci sono ulteriori elementi. `_fetch_paginated()` segue questo cursore iterativamente finché `"next"` è `null`, concatenando i risultati in un'unica lista. In modalità DB-less tutti gli oggetti entrano in una singola pagina e `"next"` è sempre `null`, quindi il ciclo termina alla prima iterazione; lo stesso meccanismo funziona correttamente in ambienti con Kong con database, dove le risorse possono essere distribuite su più pagine.

Il ciclo di vita dell'adapter si articola su due momenti distinti. In Fase 3, `check_connectivity()` verifica che la Admin API sia raggiungibile prima di popolare `TargetContext.gateway`: se la verifica fallisce, il campo rimane `None` e tutti i test `WHITE_BOX` transitano a `SKIP` per l'intera run. Durante l'esecuzione dei test (Fase 5), qualsiasi errore nelle chiamate HTTP verso l'Admin API viene incapsulato in `GatewayAdapterError`, che il test cattura e converte in `TestResult(ERROR)` senza propagare l'eccezione all'engine.

4.5.2 Degradazione Controllata a Tre Livelli (D4.P5)

Un assessment eseguito in ambienti eterogenei incontra sistematicamente situazioni in cui parti dell'infrastruttura non sono disponibili: tool esterni non installati, Admin API non esposta, integrazione con tool esterni disabilitata per policy. La risposta del sistema non è un errore a cascata, ma una degradazione controllata su tre livelli distinti che produce risultati parziali corretti con motivazioni esplicite in ogni caso.

- **Livello 1 - master switch globale.** Il flag `external_tools.enabled` in `config.yaml` disabilita l'intera gerarchia `ExternalToolTest` in un'unica operazione: l'`ExternalTestRegistry` restituisce una lista vuota senza scandagliare il filesystem né importare moduli, con zero overhead e zero `SKIP` nel log. Serve per ambienti in cui la presenza di tool esterni è impossibile per policy di sicurezza o di deployment.
- **Livello 2 - tool esterno assente.** Quando il master switch è attivo ma un singolo tool non è disponibile, la Phase R4 descritta nella Sezione 4.2.1 imposta

`_skip_reason_from_registry` su ogni test del gruppo: ogni test riceve **SKIP** con motivazione esplicita, e un solo **WARNING** viene emesso nel log per tool mancante.

- **Livello 3 - Admin API del gateway assente.** Se `gateway_adapter` non è configurato in `config.yaml` oppure se `check_connectivity()` fallisce in Phase 3, `TargetContext.gateway` rimane `None`. Ogni test `WHITE_BOX` chiama `_requires_admin_api(target)` come prima operazione e restituisce **SKIP** con testo esplicito senza eseguire alcuna logica di verifica. L'assessment copre in questo scenario i soli domini `BLACK_BOX` e `GREY_BOX`, con un numero di **SKIP** pari al numero di test `WHITE_BOX` nel set attivo.

In tutti e tre i casi, gli **SKIP** prodotti sono distinguibili nel report per motivazione (campo `skip_reason`), permettendo all'operatore di separare le garanzie non verificate per impossibilità tecnica da quelle non verificate per prerequisiti mancanti.

4.6 Evidence Store e Sanitizzazione delle Credenziali

Ogni verifica di sicurezza produce due tipi di output: un verdetto, espresso come `TestResult`, e l'evidenza che lo sostiene, composta dalle transazioni HTTP effettuate dai test nativi e dai dati grezzi prodotti dai tool esterni. L'evidenza è ciò che distingue un risultato verificabile da un'asserzione: senza di essa, un **FAIL** non può essere riprodotto né contestato. Raccoglierla introduce però una responsabilità: un audit trail non sanitizzato che contiene token o credenziali è esso stesso un rischio di sicurezza: il file viene condiviso con il team di analisi e archiviato nei sistemi di ticketing, dove la sua esposizione sfugge al controllo di chi ha eseguito l'assessment. Questa sezione descrive come l'`EvidenceStore` gestisce raccolta affidabile e sanitizzazione strutturale durante la Fase 5, e come vengono generati i tre deliverable finali dell'assessment in Fase 7 (introdotti nella Sezione 4.1): `evidence.json` per l'archiviazione forense, `assessment_report.html` per la consultazione operativa e `apiguard_report.json` per l'integrazione strutturata nei sistemi CI/CD.

4.6.1 Ciclo di Vita dello Streaming Evidence Store (D4.P1)

Il design dell'`EvidenceStore` è motivato nella Sezione 3.2.5, dove viene discusso il problema strutturale del buffer a capacità fissa che produceva audit trail rotti in silenzio. Questa sezione descrive l'implementazione concreta che ha risolto quel problema.

Il ciclo di vita dell'`EvidenceStore` si articola in quattro operazioni sincronizzate con le fasi dell'engine. In Phase 3, l'engine istanzia `EvidenceStore` e crea la directory temporanea `outputs/evidence_tmp/`. Prima di eseguire ogni test in Phase 5, l'engine chiama `store.begin_test(test_id)`: il metodo apre il file `outputs/evidence_tmp/{test_id}.jsonl`

in modalità `append` e lo mantiene aperto per l'intera durata del test. Durante l'esecuzione, ogni chiamata a `store.log_transaction()` o `store.pin_artifact()` scrive immediatamente un singolo record in formato JSON Lines (JSONL), forzando la scrittura su disco tramite `flush()` esplicito: zero buffering in memoria, zero rischio di perdita in caso di crash. Al termine del test, `store.end_test()` chiude il descrittore del file. In Phase 7, `store.merge_and_finalize()` recupera tutti i file `.jsonl` dalla directory temporanea, applica un ordinamento lessicografico basato sul `test_id` per preservare il determinismo dell'output, concatena i record in una singola collezione strutturata in formato JavaScript Object Notation (JSON) e li scrive nel file definitivo `outputs/evidence.json`. La directory temporanea viene poi rimossa.

Riportando la soluzione iniziale con `deque(maxlen=100)`, il primo record di un test con più di 100 transazioni veniva silenziosamente scartato dalla coda, generando `Finding.evidence_ref` che puntavano a record inesistenti nell'audit trail senza produrre nessun errore visibile. Il design corrente elimina questa classe di bug per costruzione: ogni record scritto su disco è permanente, e il numero di record per test è illimitato. L'unico parametro che incide sulla dimensione dell'archivio è `RESPONSE_BODY_MAX_CHARS = 10 000`: la costante tronca il body di risposta di ogni singolo record per evitare che risposte molto grandi saturino il disco, lasciando il conteggio dei record invariato.

La proprietà di recuperabilità è un effetto collaterale dell'architettura. Se il processo va in crash tra Phase 5 e Phase 7, i file `.jsonl` rimangono su disco e sono leggibili manualmente con qualsiasi strumento JSONL-compatibile: l'evidenza parziale è recuperabile anche in assenza di un run completo.

4.6.2 Sanitizzazione Centralizzata delle Credenziali (D4.P2)

Un audit trail che contiene token di autenticazione, API key o password è esso stesso una superficie di rischio: `evidence.json` viene condiviso con il team di sicurezza, archiviato nei sistemi di ticketing, e potenzialmente incluso in repository. Delegare la redazione ai singoli connector presuppone che ogni implementazione, incluse quelle di terze parti, ricordi di oscurare i propri campi sensibili: l'adozione di logiche custom e frammentate elimina la possibilità di definire garanzie uniformi, poiché ogni estensione opererebbe secondo criteri arbitrari e privi di coordinamento.

La responsabilità della redazione viene pertanto centralizzata nel punto di scrittura su disco, operando attraverso due flussi distinti in base alla natura dell'evidenza: gli artifact generati dai tool esterni e le transazioni HTTP prodotte dai test nativi. Gli artifact dei tool esterni passano per `EvidenceStore._sanitize_artifact()`, invocato automaticamente da `pin_artifact()` prima della scrittura su disco. Questo meccanismo solleva i connector dall'onere della pulizia dei dati e applica tre tecniche in sequenza sul dizionario `raw_output` grezzo :

1. **Key-pattern matching:** un'espressione regolare esamina ricorsivamente ogni chiave del dizionario rispetto a 11 pattern sensibili (`token`, `password`, `api_key`, `apikey`, `authorization`, `bearer`, `secret`, `credential`, `auth`, `private_key`, `access_token`). Il confronto richiede l'esatta corrispondenza della stringa e ignora le sottostringhe: questa scelta previene falsi positivi ed evita la redazione involontaria di campi legittimi (ad esempio, una chiave `author` non viene censurata pur contenendo la sequenza `auth`). Qualsiasi chiave con corrispondenza vede il proprio valore sostituito con `[REDACTED]`, indipendentemente dal tipo.
2. **JWT value detection:** la regex `_SANITIZE_JWT_PATTERN` intercetta le stringhe che contengono la firma di un token JWT⁷, offuscando il valore indipendentemente dal nome della chiave che lo ospita; cattura quindi i token che appaiono come valori in chiavi con nomi arbitrari.
3. **Header prefix matching:** qualsiasi stringa che inizia con `"Bearer "`, `"Basic "`, `"Token "` o `"token "` viene redactata, intercettando i token che compaiono nei valori di header HTTP inclusi nei record di evidenza.

La sovrapposizione dei tre meccanismi copre scenari che nessuno dei tre individualmente intercetterebbe: un token JWT in una chiave `"result"` arbitraria supera il primo meccanismo (nome non riconosciuto) ma viene catturato dal secondo (struttura `base64url`); un header `Authorization` in un dict annidato viene catturato dal primo meccanismo (key-pattern matching ricorsivo). Spostando il presidio di sicurezza al confine dello storage, un connector di terze parti privo di controlli interni non compromette l'integrità dell'audit trail. Il sistema diventa intrinsecamente resiliente a implementazioni difettose, poiché la garanzia risiede nel punto di consolidamento e non nel punto di generazione del dato.

I tre meccanismi appena descritti si applicano esclusivamente agli artifact dei connector; le transazioni HTTP generate dai test nativi seguono invece un percorso separato. La redazione non avviene a posteriori sul dizionario, ma interviene direttamente al livello del modello Pydantic: un `field_validator` applicato all'attributo `EvidenceRecord.request_headers` intercetta la creazione del record e sostituisce il valore dell'header `Authorization` con `[REDACTED]`. Questo approccio preventivo assicura che nessuna credenziale nativa raggiunga l'`EvidenceStore` in chiaro.

4.6.3 Dual Audit Trail e Report Domain-Centric (D5.P5)

I tre deliverable finali dell'assessment assolvono funzioni distinte e si rivolgono a interlocutori diversi. Il termine *Dual* si riferisce quindi al parallelismo tra la persistenza

⁷La struttura standard di un token JWT si compone di tre parti distinte, denominate *header*, *payload* e *signature*, codificate in Base64URL e separate da un punto.

tecnica destinata alle macchine (`evidence.json`) e la presentazione operativa destinata all'analisi umana (`assessment_report.html`).

`evidence.json` è l'archivio forense. Contiene due categorie di record unite in un'unica struttura serializzata: le transazioni HTTP che i test nativi hanno marcato come evidenza di FAIL, con il body di risposta fino a 10.000 caratteri, e gli artifact dei tool esterni, ossia gli output JSON dei connector sanitizzati e pinned tramite `pin_artifact()`. Ogni record è autosufficiente per ricostruire la probe o l'analisi che ha rivelato la vulnerabilità. Il file affianca i metadati di intestazione (`generated_at_utc`, `record_count`) alla collezione lineare dei record, creando un formato JSON nativamente compatibile con l'ingestion da parte delle piattaforme di log aggregation.

`assessment_report.html` rappresenta il deliverable operativo, renderizzato da `report/renderer.py` tramite il motore Jinja2⁸. Il documento include le statistiche aggregate, la conformità normativa e il **TransactionSummary**: la cronologia completa delle transazioni per ciascun test, che attesta la *proof of coverage*, permettendo di verificare il perimetro interrogato dal tool e di fare triage senza aprire `evidence.json`. Il report è generato come artefatto autonomo, con stili Cascading Style Sheets (CSS) e script integrati *inline* per garantirne la consultazione offline senza dipendenze esterne. Le transazioni adottano inoltre tecniche di lazy loading per mantenere la reattività dell'interfaccia su dataset massivi.

`apiguard_report.json` fornisce la serializzazione strutturata dell'oggetto `ReportData`, il quale contiene tutti i risultati, le statistiche aggregate e i metadati della run. L'architettura garantisce una rigorosa identità strutturale tra questo documento (prodotto nativamente dalla CLI) e i dati JSON scaricabili in locale dall'interfaccia HTML del report. Questa sincronizzazione assicura che il **TransactionSummary** visionato dall'utente e i dati consumati dalle pipeline CI/CD o dai sistemi Security Information and Event Management (SIEM) coincidano esattamente, azzerando il rischio di incongruenze.

L'organizzazione visiva del report riflette la classificazione operata da `report/builder.py`: all'interno di ogni dominio di assurance, i verdetti dei test nativi precedono i risultati derivati dai tool esterni, sfruttando il campo `TestResult.source`. Questa separazione guida la lettura dell'analista, isolando le verifiche protocollari dirette dall'orchestrazione di strumenti di terze parti.

⁸Una libreria di templating per Python che consente la generazione dinamica di HyperText Markup Language (HTML) a partire da strutture dati tipizzate.

Capitolo 5

Validazione sperimentale e risultati

I Capitoli 3 e 4 hanno descritto come APIGuard Assurance è stato progettato e realizzato. La suite attiva comprende test per i domini D0, D1, D2, D3, D4, D6 e D7; il Dominio 5 (Observability) è architetturalmente predisposto ma non include test attivi in questa fase, come dichiarato nella Sezione 3.3.1. Questo capitolo porta quelle scelte architetturali a confronto con un target reale, verificando empiricamente che producano i comportamenti dichiarati.

La validazione si articola in quattro livelli con una relazione di preconditione: il testbed stabilisce le condizioni di osservazione (Sezione 5.1); la qualità ingegneristica del codebase certifica che lo strumento di misura è affidabile (Sezione 5.2); la copertura funzionale e l'idempotenza dimostrano che i risultati sono stabili e privi di effetti collaterali (Sezione 5.3); il profilo computazionale ne qualifica l'integrabilità operativa (Sezione 5.4).

Tutti i dati quantitativi di questo capitolo provengono da verifiche condotte sulla versione 0.1.0 del tool: misurazioni di sistema, esecuzioni ripetute dell'assessment e ispezioni statiche del codebase, ciascuna con una procedura o un comando riproducibile riportato nel testo.

5.1 Topologia del Testbed Sperimentale

La scelta dell'ambiente target è derivata dalla matrice di copertura della suite: serviva un'applicazione con OpenAPI Specification (OAS) nativa non generata a posteriori, con endpoint protetti da autenticazione reale, almeno due livelli di privilegi distinti, endpoint parametrici con valori di path risolvibili, e comportamenti osservabili e stabili attraverso run successivi senza modifiche al target. Forgejo 14.0.3, esposto attraverso Kong 3.9 in

modalità DB-less, soddisfa tutti questi requisiti. La scelta conferma inoltre in modo concreto l'agnosticismo applicativo discusso nella Sezione 3.1.1: il tool non contiene nessun riferimento specifico a Forgejo: cambiare target significa aggiornare i valori nel `config.yaml` (URL del gateway, percorso della specifica, path seed, credenziali), non modificare il codice.

5.1.1 Composizione e Connettività dell'Ambiente di Test

L'ambiente di test è composto da quattro servizi che comunicano all'interno di un network Docker dedicato (**lab-net**): Forgejo¹ 14 come applicazione target, Kong 3.9 in modalità DB-less come gateway che riceve e instrada tutto il traffico verso Forgejo, PostgreSQL 15-alpine come backend di persistenza richiesto da Forgejo, e un container **forgejo-setup** che crea gli account al primo avvio e poi termina. Il traffico del tool parte dall'host, raggiunge Kong attraverso le porte mappate sul sistema, e Kong lo inoltra a Forgejo attraverso un upstream dichiarato nel file di configurazione `kong.yaml`. L'intero ambiente è dichiarato in un singolo file Docker Compose ed eseguito su host di sviluppo, senza hardware dedicato né virtualizzazione separata. La Tabella 5.1 riporta le versioni fissate.

Kong espone tre porte sull'host: la porta 8000 per il traffico HTTP semplice, usata dal test 1.5 per verificare il redirect verso HTTPS; la porta 8443 per il traffico HTTPS con certificato self-signed generato localmente, che costituisce l'endpoint primario per tutti gli altri test; la porta 8001 per la Admin API, che fornisce i dati di configurazione ai test **WHITE_BOX** dei Domini 3, 4 e 6. La configurazione di Kong è interamente contenuta nel file `kong.yaml`, montato in sola lettura nel container: Kong legge il file a ogni avvio e opera senza database. Questo pattern, detto DB-less, è rilevante per la validazione: la configurazione è interamente nel file versionato `kong.yaml` e non può essere modificata a runtime tramite l'Admin API, il che garantisce che lo stato del gateway sia identico a ogni avvio e che le verifiche **WHITE_BOX** interrogino sempre la stessa configurazione. Kong supporta anche la modalità con database, ma in quel caso la configurazione potrebbe essere alterata tra una run e l'altra.

5.1.2 Configurazione del Testbed per la Validazione Funzionale

Validare un tool di assessment su un ambiente completamente configurato in modo corretto produrrebbe una suite con tutti **PASS**: un risultato tautologico che non dimostra nulla sulla capacità del tool di rilevare violazioni reali. Il testbed è configurato perché

¹Forgejo è un servizio di hosting Git self-hosted, derivato da Gitea, con API REST documentata tramite specifica OpenAPI nativa. Espone repository, utenti, token e webhook tramite un'interfaccia HTTP standardizzata.

Componente	Versione	Ruolo nel testbed
apiguard-assurance	0.1.0	Tool sotto validazione
Python	3.12.3	Interprete del tool
Forgejo	14.0.3 (gitea-1.22.0)	Target applicativo: API REST con specifica OpenAPI nativa
PostgreSQL	15-alpine	Backend di persistenza di Forgejo (dipendenza funzionale, non componente autonomo)
Kong	3.9 DB-less	API Gateway: proxy HTTP/HTTPS, instradamento verso Forgejo, Admin API su porta 8001
nuclei	3.8.0	Tool esterno per Shadow API discovery (test <code>ext.0.1</code>)
nuclei-templates	10.4.3	Base di template di scansione versioned: garantisce che lo stesso set di firme venga usato in ogni run
testssl.sh	3.2.3	Analisi TLS black-box (test <code>ext.1.5.testssl</code>)
sslyze	(libreria Python)	Analisi TLS via libreria Python (test <code>ext.1.5.sslyze</code>)

Tabella 5.1: Componenti e versioni dell'ambiente di test.

alcuni test producano **FAIL** attesi e altri **PASS** attesi; la combinazione costituisce un banco di prova funzionalmente completo in cui il tool deve saper distinguere i due casi.

Le condizioni che producono **FAIL** osservabili e attesi nell'assessment sono le seguenti:

- Il plugin `rate-limiting` di Kong è commentato nel file `kong.yml`. Il test 4.1, che verifica la presenza di un limite di velocità inviando richieste in loop fino a ricevere HTTP 429, registra **FAIL** con 2 finding. La disattivazione del rate limiting è una condizione ricorrente in ambienti non produttivi dove il throttling interferirebbe con i test applicativi, e costituisce perciò uno scenario realistico.
- La specifica OpenAPI di Forgejo dichiara a livello globale un requisito di autenticazione tramite il campo `security` del documento. Questo marcatore viene propagato a tutti gli endpoint dall'analizzatore della specifica, che li classifica come richiedenti autenticazione. Alcuni di quegli endpoint sono però genuinamente pubblici: quando il test 1.1 li interroga senza credenziali e riceve **2xx**, l'oracle a tre esiti della Sezione 4.3.5 classifica il risultato come **BYPASSED** e produce un finding. Il verdetto è **FAIL** con 74 finding. Il comportamento del tool è corretto: la discrepanza tra ciò che la specifica dichiara e ciò che il target restituisce è esattamente la classe di anomalia che il contract-driven testing è progettato per rilevare. Questo caso illustra in modo

concreto la dipendenza dall'oracolo descritta nella Sezione 3.1.2: quando la specifica dichiara un requisito che il target non applica completamente, i finding prodotti sono corretti dal punto di vista contrattuale ma richiedono che l'analista valuti se la discrepanza sia una configurazione intenzionale o una vulnerabilità effettiva.

- Il testbed usa un certificato TLS autofirmato generato localmente tramite lo script `certs/gen-certs.sh`. Questa è una scelta deliberata: un certificato autofirmato presenta per costruzione configurazioni al di sotto degli standard raccomandati per la produzione, suite di cifratura, durata e catena di certificazione incluse, e costituisce uno scenario realistico per ambienti di sviluppo e staging. I test `ext.1.5.sszlyze` e `ext.1.5.testssl` rilevano rispettivamente 1 e 3 finding sulla configurazione del protocollo, verificando che l'analisi TLS del Dominio 1 sappia distinguere le singole debolezze e produrre finding specifici per ciascuna.

Le configurazioni che producono invece PASS attesi sono:

- Il plugin `response-transformer` inietta gli header di sicurezza HTTP su tutte le risposte: `Strict-Transport-Security`, `Content-Security-Policy`, `X-Content-Type-Options` e `X-Frame-Options`. Il test 0.1 verifica la presenza di questi header e registra PASS.
- Il parametro `KONG_HEADERS=off` rimuove l'header `Server: kong/version` da tutte le risposte. Il test 6.2, che verifica l'assenza di fingerprinting del gateway, registra PASS.
- Il servizio `http-redirect-service` produce un redirect 301 con header `Location` corretto per qualsiasi richiesta HTTP sulla porta 8000. Il test 1.5, che verifica il redirect HTTP→HTTPS, registra PASS.

5.1.3 Struttura RBAC e Provisioning degli Utenti

Il container `forgejo-setup` crea tre account al primo avvio: `thesis-admin` (amministratore), `user-a` (utente con privilegi ordinari) e `user-b` (secondo utente non privilegiato). La struttura a tre livelli è il minimo necessario per la matrice di test: `thesis-admin` è il soggetto dei test di autenticazione che richiedono token con privilegi elevati; `user-a` è il soggetto del test 2.1 (RBAC enforcement), che tenta endpoint amministrativi con un token non-admin e attende 403; `user-b` è necessario per i test di isolamento tra oggetti di utenti distinti, che verificano che le risorse di `user-a` non siano accessibili con il token di `user-b`. I test 1.4 e 2.1 dipendono da token validi prodotti dal test 1.1, dipendenza dichiarata nel DAG come vincolo di Phase B e discussa nella Sezione 5.4.

Il provisioning è deterministico. Il container `forgejo-setup` usa l'interfaccia CLI di Forgejo con flag `--or-true`, così da non fallire se gli account esistono già da un avvio precedente. Questa scelta realizza concretamente la proprietà di riproducibilità discussa nella Sezione 3.1.4: il testbed raggiunge lo stesso stato iniziale indipendentemente da quante volte viene avviato, condizione necessaria perché i run successivi siano comparabili.

5.2 Qualità della Codebase come Prerequisito di Credibilità

Un tool che produce verdetti di sicurezza deve poter dimostrare che la propria implementazione risponde agli stessi criteri di qualità che applica nell'analisi del target. Una codebase con errori di tipo non rilevati può produrre verdetti formalmente corretti ma basati su dati mal classificati; dipendenze con Common Vulnerabilities and Exposures (CVE) note espongono il tool stesso ad attacchi che potrebbero alterarne i risultati; simboli pubblici privi di contratto documentato rendono l'estensione da parte di terzi una fonte di errori non rilevabili staticamente. La credibilità dell'output dipende dalla qualità del codice che lo genera, e questa sezione la stabilisce prima di discutere cosa il tool ha trovato sul target.

Le verifiche coprono tre aree: l'analisi statica della codebase tramite una suite di strumenti automatizzati (Sezione 5.2.1), la type safety a due strati che combina analisi statica con mypy e validazione a runtime con Pydantic (Sezione 5.2.2), e gli indicatori di qualità che riguardano documentazione interna, riproducibilità della build e portabilità dell'installazione (Sezione 5.2.3).

5.2.1 Analisi Statica della Codebase

La suite di analisi statica della codebase è composta da cinque strumenti con responsabilità distinte e complementari, i cui risultati sono riassunti nella Tabella 5.2:

- **ruff** svolge l'analisi di lint², l'ordinamento degli import, il rilevamento di pattern insicuri e la verifica della copertura delle annotazioni di tipo.
- **mypy** verifica la coerenza dei tipi lungo l'intera catena di dipendenze in modalità **strict**, la configurazione più restrittiva disponibile. Il suo funzionamento è discusso in dettaglio nella Sezione 5.2.2.
- **bandit** analizza la codebase alla ricerca di pattern associati a vulnerabilità note. I tre finding di bassa severità rilevati in `connectors/base.py` riguardano l'invo-

²Con *lint* si intende l'analisi statica automatica del codice sorgente volta a identificare errori, violazioni di stile, pattern rischiosi e codice non conforme alle convenzioni del progetto, senza eseguire il programma.

cazione di `subprocess.run()` e sono soppressi con annotazione documentata: il comando è costruito esclusivamente da `ClassVar` dichiarati nel sorgente e da valori già validati da Pydantic, senza mai includere input HTTP grezzo dall'esterno.

- **vulture** identifica simboli pubblici definiti ma mai invocati direttamente nel codice, con soglia di confidenza all'80%. I metodi `execute()`, `model_*` e `validator_*` sono esclusi tramite whitelist: `execute()` è invocato dall'engine tramite `isinstance` check, mentre i validator Pydantic vengono attivati automaticamente dal framework al momento della costruzione di ogni oggetto, senza mai comparire come chiamate esplicite nel sorgente.
- **pip-audit** verifica le dipendenze dichiarate contro i database pubblici di vulnerabilità note.

Tool	Esito	Note
ruff	0 errori	Ruleset E/W/F/I/N/UP/B/S/ANN
mypy	0 errori	Modalità <code>strict</code> : <code>no-implicit-optional</code> , <code>disallow-untyped-defs</code> , <code>warn-return-any</code>
bandit	0 finding severity medium+	3 Low (B404, S603×2) soppressi con <code># noqa</code> documentati: invocazione subprocess controllata per design
vulture	0 finding	Confidence $\geq 80\%$; metodi a dispatching dinamico esclusi via whitelist
pip-audit	0 CVE	Nessuna vulnerabilità nota nelle dipendenze dichiarate

Tabella 5.2: Risultati dell'analisi statica su APIGuard Assurance v0.1.0.

L'analisi statica con mypy verifica che i tipi dichiarati tra moduli siano consistenti: se `execute()` dichiara di restituire `TestResult` e un punto di chiamata la tratta come `str`, mypy lo intercetta prima dell'esecuzione. Ciò che mypy non copre è il confine con l'esterno: un file `config.yaml` caricato da disco o l'output grezzo di un connector esterno entrano nel sistema come strutture non ancora tipizzate, e nessuna verifica statica può garantirne la forma. Quel secondo livello è trattato nella Sezione 5.2.2.

5.2.2 Dual-Layer Type Safety (D6.P3)

La type safety in un codebase Python ha due confini distinti con caratteristiche diverse. Il primo è interno: i contratti tra moduli, espressi tramite type hints, verificabili staticamente prima dell'esecuzione. Il secondo è esterno: il punto in cui dati non tipizzati provenienti dall'esterno, un file `config.yaml` caricato da disco o il raw output di un connector, entrano nel sistema. I due confini richiedono meccanismi diversi.

mypy in modalità **strict** verifica il confine interno sull'intera cartella **src/**: controlla che ogni funzione dichiari esplicitamente i tipi di ingresso e uscita (**disallow-untyped-defs**), che nessun parametro ammetta **None** implicitamente (**no-implicit-optional**), e che nessuna funzione restituisca **Any** dove è atteso un tipo concreto (**warn-return-any**). Questo include i modelli in **src/core/models/** (**TestResult**, **Finding**, **ConnectorResult**, **AttackSurface** e gli altri), le classi base dei test e dei connector, e tutta la logica dell'engine. Se durante lo sviluppo un campo di **TestResult** venisse rinominato o rimosso, ogni punto del codebase che vi accede genererebbe un errore mypy immediatamente, prima che il codice venga mai eseguito; allo stesso modo, se un tipo di ritorno dichiarato non corrisponde a ciò che la funzione restituisce effettivamente, mypy lo segnala al momento della modifica.

Pydantic v2 presidia il confine esterno. Ogni struttura dati che attraversa il confine del sistema, dai campi di **config.yaml** fino agli oggetti **ConnectorResult** prodotti dai tool esterni, è modellata come classe Pydantic con validatori espliciti. I **model_validator** e i **field_validator** applicano le invarianti nel momento della costruzione dell'oggetto: se nel **config.yaml** un tool esterno è marcato **enabled = True** ma il campo **timeout_seconds** è assente, la Phase 1 solleva un **ConfigurationError** con il percorso esatto del campo mancante e l'assessment non inizia. Se un connector produce un **ConnectorResult** con un campo obbligatorio mancante, Pydantic solleva un errore di validazione al momento della costruzione dell'oggetto, indicando il campo violato, prima che il dato raggiunga la logica di assessment.

I due strati hanno copertura complementare: mypy intercetta gli errori di interfaccia tra moduli prima dell'esecuzione, Pydantic intercetta i dati malformati al confine con l'esterno al momento della costruzione. Un difetto che sfugge al primo viene catturato dal secondo; la copertura combinata riduce lo spazio degli errori che possono propagarsi in silenzio verso la logica di assessment. Per chi aggiunge un test o un connector, entrambi i meccanismi sono già attivi per costruzione: i modelli Pydantic di configurazione e di risultato sono già frozen e tipizzati, e la conformità ai type hints è verificabile con **mypy src/** senza configurazione aggiuntiva.

5.2.3 Indicatori di Qualità dell'Ingegneria di Produzione

Tre verifiche ulteriori riguardano la riproducibilità e la portabilità del tool come artefatto distribuibile, indipendentemente dall'output dell'assessment.

La prima è la copertura della documentazione interna: tutti i 292 simboli pubblici della codebase presentano una docstring, verificato tramite ispezione dell'Abstract Syntax Tree (AST)³. In un tool progettato per essere esteso, la documentazione dei contratti pubblici

³L'Abstract Syntax Tree è la rappresentazione strutturata del codice sorgente usata dai compilatori e dagli analizzatori statici. Ispezionare l'AST permette di estrarre i simboli pubblici definiti e verificare

è il meccanismo che permette a chi aggiunge un test di determinare cosa può chiamare e con quali aspettative di comportamento.

La seconda è la riproducibilità della build: due esecuzioni consecutive di `hatch build --target wheel`⁴ producono un artefatto con hash SHA-256 identico (451 261 byte). Una build non deterministica introdurrebbe variabilità tra versioni nominalmente identiche, rendendo impossibile garantire che il tool in produzione corrisponda esattamente all'artefatto verificato.

La terza riguarda l'installabilità in ambienti puliti. Il pacchetto generato dalla build può essere installato in un ambiente virtuale Python privo di dipendenze preinstallate con `pip install`; al termine, `apiguard version` restituisce 0.1.0 e il tool è operativo. La verifica conferma che l'installazione non richiede dipendenze di sistema non dichiarate e che il comportamento del tool non dipende da pacchetti preinstallati nell'ambiente dell'esecutore, condizione descritta nella Sezione 3.2.6.

Aspetto	Stato	Dettaglio
Analisi statica	✓	0 errori su tutti e cinque gli strumenti (Sezione 5.2.1)
Type safety	✓	0 errori mypy strict; validazione Pydantic attiva su tutti i confini (Sezione 5.2.2)
Documentazione interna	✓	292/292 simboli pubblici con docstring (100%)
Riproducibilità della build	✓	SHA-256 identico su due build consecutive (451 261 byte)
Portabilità	✓	Cold install in ambiente pulito: <code>apiguard version</code> → 0.1.0

Tabella 5.3: Sintesi delle verifiche di qualità sulla codebase.

5.3 Copertura Funzionale e Idempotenza dell'Assessment

I test che creano risorse sul target, le interrogano e poi le eliminano introducono dipendenze di stato che, se mal gestite, producono risultati diversi tra un'esecuzione e la successiva: la prima modifica il target, la seconda trova uno stato alterato. Dimostrare empiricamente che due esecuzioni successive producono risultati identici è il modo concreto per verificare che la pipeline non lasci effetti collaterali osservabili e che le proprietà di teardown e riproducibilità, discusse nelle Sezioni 4.3.8 e 3.1.4, si comportino nel modo dichiarato su un target reale.

la presenza della docstring associata senza eseguire il programma.

⁴`hatch` è il build backend del progetto. Il comando produce un file `.whl` (*wheel*), il formato standard di distribuzione per i package Python: installabile con `pip` senza compilazione, non dipende dalla piattaforma e non richiede estensioni native. Il tag `py3-none-any` nel nome lo conferma.

Le due esecuzioni sono state condotte in sequenza nella stessa giornata contro lo stesso testbed, senza alcun intervento manuale tra l'una e l'altra. Questa sezione analizza i risultati per dominio (Sezione 5.3.1) e verifica la stabilità dell'esecuzione e il corretto cleanup delle risorse (Sezione 5.3.2).

KPI	Run 1	Run 2	Δ
PASS / FAIL / SKIP / ERROR	9 / 7 / 2 / 0	9 / 7 / 2 / 0	identico
Finding count totale	98	98	identico
pass_rate_pct	56.2%	56.2%	identico
assessment_duration_seconds	290.14 s	290.81 s	+0.23%
Exit code / label	1 / FAIL	1 / FAIL	identico

Tabella 5.4: KPI di idempotenza su due run indipendenti.

5.3.1 Analisi dei Risultati per Dominio

L'assessment ha prodotto 98 finding distribuiti su 7 FAIL, 9 PASS e 2 SKIP, riportati per test nella Tabella 5.5. Prima di analizzare i risultati per dominio, è utile presentare una lettura trasversale dei tre esiti.

I 7 FAIL, con il dettaglio dei finding per test nella Tabella 5.5, corrispondono alle condizioni configurate nel testbed descritte nella Sezione 5.1.2. Il test 1.1 da solo produce 74 dei 98 finding totali (75.5%): quegli endpoint di Forgejo che il campo `security` della specifica marca come autenticati, ma che rispondono 2xx a richieste prive di credenziali, sono finding corretti dal punto di vista del contratto tra specifica e comportamento osservato, come discusso nella Sezione 5.1.2. I restanti 24 finding sono distribuiti su test 0.1 (16), test 0.2 (1), test 4.1 (2), test 7.2 (1), `ext.1.5.sslyze` (1) e `ext.1.5.testssl` (3).

I 2 SKIP illustrano il comportamento del sistema in assenza di condizioni di applicabilità. Il contratto di `execute()`, descritto nella Sezione 4.3, stabilisce che ogni test produce sempre un `TestResult`: quando lo scope applicabile è vuoto o un prerequisito non è soddisfatto, il risultato è SKIP con motivazione esplicita, non un errore silenzioso. Il test 0.3 verifica che gli endpoint marcati `deprecated: true` nella specifica restituiscano 410 Gone o portino un header `Sunset` (RFC 8594); la specifica OpenAPI di Forgejo 14.0.3 non dichiara nessun endpoint deprecato, e il test restituisce SKIP con motivo documentato. Il test 1.6 verifica la configurazione del session store tramite la Admin API di Kong; nella configurazione del testbed Kong non gestisce un session store esterno, e la condizione non è soddisfatta. In entrambi i casi uno SKIP con motivazione esplicita è la risposta semanticamente corretta, distinta da un ERROR che indicherebbe un malfunzionamento della pipeline.

Test ID	Dominio	Tipo	Status	Finding
0.1	D0: Discovery	Nativo	FAIL	16
0.2	D0: Discovery	Nativo	FAIL	1
0.3	D0: Discovery	Nativo	SKIP	0
1.1	D1: Auth	Nativo	FAIL	74
1.4	D1: Auth	Nativo	PASS	0
1.5	D1: Auth	Nativo	PASS	0
1.6	D1: Auth	Nativo	SKIP	0
2.1	D2: Authorization	Nativo	PASS	0
3.3	D3: Data Integrity	Nativo	PASS	0
4.1	D4: Availability	Nativo	FAIL	2
4.2	D4: Availability	Nativo	PASS	0
4.3	D4: Availability	Nativo	PASS	0
6.2	D6: Hardening	Nativo	PASS	0
6.4	D6: Hardening	Nativo	PASS	0
7.2	D7: Business Logic	Nativo	FAIL	1
ext.0.1.nuclei	D0: Discovery	Esterno	PASS	0
ext.1.5.sslyze	D1: Auth	Esterno	FAIL	1
ext.1.5.testssl	D1: Auth	Esterno	FAIL	3
Totale				98

Tabella 5.5: Status e finding count per test.

I 9 PASS coprono le configurazioni correttamente applicate nel testbed: l'assenza di fingerprinting del gateway (test 6.2), il redirect HTTP→HTTPS (test 1.5), la revoca corretta del token (test 1.4), l'enforcement dell'autenticazione sui test GREY_BOX (test 2.1), la configurazione degli upstream Kong (test 6.4), i controlli di integrità dei dati e disponibilità (test 3.3, 4.2, 4.3), e il tool esterno `ext.0.1.nuclei` che non ha rilevato Shadow API, confermando che la superficie documentata dalla specifica corrisponde a quella effettivamente esposta.

5.3.2 Idempotenza e Validazione del Teardown

La Tabella 5.4 riporta i KPI aggregati delle due esecuzioni. I contatori di finding, gli status per-test, il codice di uscita e il `pass_rate_pct` sono byte-identici tra i due run: la parte deterministica del risultato non mostra alcuna variazione. Il delta sul wall-clock è di 0.67 secondi su una durata di circa 290 secondi, pari allo 0.23% della durata totale; questa variazione rientra nella fluttuazione attesa delle latenze di rete HTTP verso il target e non è attribuibile a comportamento non deterministico della pipeline.

Il teardown, descritto nella Sezione 4.3.8, costituisce la preconditione dell'idempotenza: senza un cleanup completo delle risorse create durante la prima esecuzione, la seconda troverebbe uno stato alterato e produrrebbe risultati diversi. La verifica condotta dopo ciascuna delle due esecuzioni ha confermato che la Phase 6 ha drenato il registro LIFO senza lasciare risorse residue: nessun token con prefisso "apiguard" sull'account `thesis-admin` e nessuna repository con prefisso "apiguard" sull'account `user-a` erano presenti al termine di ciascuna esecuzione. Zero risorse residue su due esecuzioni indipendenti è la conferma empirica che la pipeline si comporta nel modo dichiarato in presenza di stato reale.

5.4 Footprint Computazionale e Benchmarking del DAG

L'integrabilità di un tool di assessment in una pipeline CI/CD dipende anche dal suo profilo di risorse: durata dell'esecuzione, consumo di memoria e dimensione degli output determinano su quale classe di infrastruttura il tool è operativamente sostenibile. Questa sezione riporta le misurazioni ottenute su due run di validazione.

Le metriche di sistema riportate nella Tabella 5.6 sono state ottenute tramite `/usr/bin/time -v` applicato al processo `apiguard run`. Il *wall-clock time* è il tempo reale trascorso dall'avvio alla terminazione, comprensivo di attese di rete, I/O e scheduling del sistema operativo. Lo *user time* e il *system time* misurano il tempo CPU consumato rispettivamente in user space (codice Python e librerie) e in kernel space (system call, gestione dei processi figli): entrambi sono prodotti da `/usr/bin/time -v` e la loro somma dà il tempo CPU totale effettivo.

Metrica	Run 1	Run 2
Wall-clock elapsed	290.14 s (4:50.14)	290.81 s (4:50.81)
User time	35.09 s	34.67 s
System time	41.51 s	41.62 s
Peak resident set size	287 MB	295 MB

Tabella 5.6: Metriche di sistema sui due run.

5.4.1 Regime di Esecuzione: Dominato dalla Latenza di Rete

La relazione tra user time, system time e wall-clock rivela il regime di esecuzione. Su 290 s di wall-clock, il processo ha consumato circa 35 s di tempo CPU in user space e 41 s in kernel space: la somma è 76 s, il 26% del totale. I restanti 214 s il processo li ha trascorsi in attesa delle risposte HTTP dal target, del completamento dei processi esterni (nuclei,

testssl.sh) e dell'I/O su disco per la scrittura dell'evidence store. Il collo di bottiglia è la latenza di rete verso il target; ottimizzazioni al codice Python avrebbero un impatto marginale sul wall-clock totale. La riduzione più significativa del tempo di esecuzione verrebbe dalla parallelizzazione dei test privi di dipendenze reciproche, direzione discussa come sviluppo futuro nella Sezione 6.3.1. Con l'architettura sequenziale attuale, il profilo misurato rappresenta il tempo di esecuzione caratteristico per un assessment di questa dimensione.

Il picco di memoria di 287–295 MB e il footprint su disco di 4.5 MB (`apiguard_report.json` 2.26 MB, `assessment_report.html` 2.0 MB, `evidence.json` 254 KB) sono compatibili con le limitazioni tipiche dei sistemi di CI/CD, che operano generalmente su macchine con vincoli di memoria nell'ordine dei gigabyte. La relazione tra topologia del DAG e footprint è discussa nella Sezione 5.4.2.

5.4.2 Topologia del DAG e Implicazioni sull'Ordinamento dell'Esecuzione

Eseguire tutti i test in un singolo batch senza DAG sarebbe la struttura più semplice: meno codice, nessun overhead di scheduling. Il problema è la correttezza dei risultati in presenza di dipendenze tra test.

Alcuni test richiedono che altri test abbiano già prodotto un certo stato nel `TestContext` prima di poter essere eseguiti. Nell'assessment di validazione, i test 1.4 (revoca del token) e 2.1 (enforcement RBAC) dipendono entrambi da un token valido prodotto dal test 1.1: senza quel prerequisito, i due test produrrebbero risultati che dipendono dall'ordine di esecuzione e non dal comportamento effettivo del target. Il DAG risolve questa dipendenza per costruzione: i test con `depends_on` dichiarato vengono schedulati in una fase successiva, e non possono mai essere avviati prima che il predecessore abbia prodotto un risultato.

La Tabella 5.7 riporta la topologia risultante: 16 test senza dipendenze formano la Phase A; 1.4 e 2.1 vengono schedulati in Phase B e non possono mai essere avviati prima che il test 1.1 abbia prodotto un risultato.

Fase DAG	Cardinalità	Test
Phase A (nessuna dipendenza)	16 test	0.1, 0.2, 0.3, 1.1, 1.5, 1.6, 3.3, 4.1, 4.2, 4.3, 6.2, 6.4, 7.2, <code>ext.0.1.nuclei</code> , <code>ext.1.5.testssl</code> , <code>ext.1.5.sslyze</code>
Phase B (<code>depends_on = ["1.1"]</code>)	2 test	1.4, 2.1

Tabella 5.7: Topologia del DAG nell'assessment di validazione.

Il DAG di validazione è sparso: su 18 nodi, solo 2 hanno dipendenze dichiarate, entrambe convergenti sullo stesso predecessore. Con questo grado di connessione il calcolo dell'ordinamento topologico è istantaneo; la giustificazione del componente non è la complessità del caso attuale, ma la scalabilità: quando la suite crescerà a coprire più domini con dipendenze incrociate tra test di aree diverse, il grafo diventerà più denso e garantire la correttezza dell'ordinamento senza un meccanismo formale diventerà progressivamente più difficile. La validazione condotta non dimostra solo che le due dipendenze dichiarate sono gestite correttamente, ma che l'infrastruttura di scheduling è funzionante e pronta per accogliere grafi più complessi, come descritto tra gli sviluppi futuri nella Sezione 6.3.1, senza modifiche architetturali al core.

La topologia ha una relazione diretta con il footprint di memoria. Con 16 test in esecuzione sequenziale nella Phase A, il `EvidenceStore` mantiene aperto un file `.jsonl` per volta: il picco di memoria non è determinato dal numero di test paralleli ma dalla dimensione del singolo audit trail. I 287–295 MB misurati sono il profilo effettivo di questa esecuzione. La transizione verso l'esecuzione parallela dei test all'interno della stessa fase, discussa come sviluppo futuro nella Sezione 6.3.1, richiederà una rivalutazione di questo profilo: più file `.jsonl` aperti concorrentemente comportano un footprint proporzionalmente maggiore, da misurare prima di fissare i requisiti di memoria per deployment CI/CD.

Sintesi della Validazione

Su un target reale con specifica OAS nativa, APIGuard Assurance v0.1.0 ha eseguito 18 test distribuiti su 7 domini di sicurezza, prodotto 98 finding, restituito risultati byte-identici su due run indipendenti, drenato tutte le risorse transitorie senza residui, e completato l'assessment in circa 4 minuti e 50 secondi. I quattro livelli di analisi di questo capitolo formano una gerarchia di precondizioni: ciascuno è condizione di interpretabilità del successivo, dal testbed che stabilisce le condizioni di osservazione fino al profilo computazionale che ne qualifica l'integrabilità operativa.

La validazione empirica misura comportamenti osservabili su un target reale; i limiti strutturali del paradigma adottato e le tensioni architetturali che ne derivano sono l'oggetto del Capitolo 6.

Capitolo 6

Discussione e Sviluppi Futuri

Il Capitolo 5 ha verificato che APIGuard Assurance produce i risultati dichiarati su un target reale; rimane aperta una questione di portata diversa, ovvero entro quali confini strutturali quella risposta vale e come il sistema si inserisce in contesti d'uso più ampi di quello sperimentale.

Questo capitolo affronta tale questione in tre direzioni. La prima è critica: i limiti strutturali del paradigma contract-driven, ciascuno radicato in una scelta architetturale specifica, vengono esaminati senza attenuazioni (Sezione 6.1). La seconda è applicativa: le condizioni concrete in cui il tool è integrabile in pipeline CI/CD industriali e come la variabilità di privilegio nei deployment reali influisce sulla copertura (Sezione 6.2). La terza riguarda le traiettorie future, distinte in estensioni operative che seguono direttamente dall'architettura corrente e in direzioni di ricerca che richiedono validazione sperimentale indipendente (Sezione 6.3).

6.1 Analisi Critica e Limiti del Paradigma Contract-Driven

Ogni scelta architetturale che garantisce un vantaggio specifico impone, per costruzione, una classe di situazioni in cui quel vantaggio diventa vincolo. Tre limiti strutturali emergono dall'analisi: la dipendenza dalla qualità della specifica OpenAPI come oracolo (Sezione 6.1.1), il confine intrinseco dell'astrazione del gateway (Sezione 6.1.2), e la dipendenza dall'accesso all'Admin API nei deployment cloud managed (Sezione 6.1.3).

6.1.1 Il Paradosso del Ground Truth (D3.P2)

Come stabilito nella Sezione 3.1.2, la specifica OpenAPI è l'unico oracolo formale della pipeline: delimita la superficie di attacco, guida la costruzione delle probe e fornisce il

contratto rispetto al quale le risposte vengono interpretate. Il vantaggio è la precisione semantica: conoscendo il contratto, il tool costruisce richieste che raggiungono la logica di sicurezza del target invece di fermarsi alla validazione sintattica dell'input. Questo è il *combinatorial wall* descritto nella Sezione 2.4.2, che il paradigma contract-driven supera usando la specifica formale come mappa della superficie di attacco.

Il prezzo è la dipendenza dalla qualità di ciò che si assume come oracolo. Una specifica che non riflette endpoint aggiunti dopo l'ultima pubblicazione, o che esclude deliberatamente endpoint sensibili dalla documentazione, restringe la superficie di attacco osservabile senza che il sistema possa rilevarlo: il tool non dispone di un meccanismo per distinguere un endpoint assente dalla specifica da uno assente dal sistema. La correttezza dell'assessment è in entrambi i casi condizionale alla correttezza del contratto; lo *specification drift* tra documentazione e implementazione reale è riconosciuto nella letteratura sul contract-driven testing come una delle cause principali di copertura incompleta, indipendentemente dallo strumento adottato. La traiettoria di ricerca che affronta strutturalmente questo paradosso è discussa nella Sezione 6.3.2.

6.1.2 La Tensione dell'Astrazione (D1.P6)

Come descritto nella Sezione 3.2.4, `BaseGatewayAdapter` è l'interfaccia che disaccoppia i test `WHITE_BOX` dal gateway concreto installato nel deployment: aggiungere il supporto per un nuovo prodotto richiede una nuova implementazione concreta senza modificare i test che la usano. Il prezzo di questa portabilità è che l'interfaccia può esporre solo ciò che è concettualmente generalizzabile a tutti i gateway che la implementano; controlli che dipendono da funzionalità vendor-specific, prive di equivalente strutturale in altri prodotti, rimangono fuori dal perimetro dell'astrazione.

Kong, ad esempio, supporta l'ordinamento esplicito dei plugin tramite un campo della sua Admin API: un plugin di autenticazione eseguito dopo un plugin di trasformazione del payload riceve dati già alterati, con implicazioni concrete sulla sicurezza. Verificare questa condizione richiede metadati specifici di Kong che non hanno corrispondente in altri gateway; includerli nell'interfaccia comune significherebbe vincolarla a una semantica vendor-specific. Il design attuale mantiene l'interfaccia generalizzabile e accetta questo come limite strutturale, non come dimenticanza. Una possibile direzione di sviluppo per superare questa tensione è descritta nella Sezione 6.3.1.

6.1.3 La Dipendenza dall'Admin API (D4.P5)

La degradazione controllata descritta nella Sezione 4.5.2 garantisce che l'assenza della Admin API non blocchi la pipeline, ma non elimina le conseguenze sulla copertura. In ambienti cloud managed, come Amazon API Gateway, Azure API Management (APIM)

o Google Cloud Endpoints, il piano di configurazione non è esposto tramite un'Admin API interrogabile direttamente: è accessibile solo attraverso le API del cloud provider, con modelli di autenticazione specifici per piattaforma. Configurare `gateway_adapter` per questi ambienti richiede un adapter dedicato per ciascun provider, attualmente non implementato.

La copertura si riduce ai domini `BLACK_BOX` e `GREY_BOX`: D0, D1, D2, D3, D4 parzialmente, D7. Il Dominio 6 (Configuration & Hardening) e le verifiche `WHITE_BOX` di D4 restano escluse. La riduzione è documentata nel report: ogni `SKIP` riporta la motivazione esplicita, rendendo la copertura effettiva immediatamente leggibile.

Questa dipendenza si manifesta in forme diverse a seconda del contesto di deployment. Nei deployment cloud managed, il piano di configurazione del gateway è accessibile solo attraverso le API del cloud provider, con modelli di autenticazione specifici per piattaforma e senza un'Admin API interrogabile direttamente: superare questo limite richiede l'implementazione di adapter dedicati, identificata come estensione operativa nella Sezione 6.3.1. In contesti di assessment formale con accesso privilegiato concordato, dove il team di sicurezza ottiene credenziali read-only con scope limitato alle risorse del progetto, la copertura `WHITE_BOX` è già raggiungibile senza modifiche al codice: è sufficiente configurare `gateway_adapter` con le credenziali e l'endpoint appropriati, a condizione che l'adapter per quel provider esista.

6.2 Applicabilità Industriale e Integrazione DevSecOps

Il tool è stato progettato per operare in contesti di integrazione continua, non solo come strumento di assessment isolato. Tre proprietà rendono concreta questa integrazione: la riproducibilità deterministica, che garantisce risultati comparabili tra esecuzioni successive; la degradazione controllata, che assicura che l'assenza di precondizioni produca output documentati senza bloccare la pipeline; e la semantica degli exit code, che fornisce un canale di comunicazione macchina-leggibile con il sistema che ospita il tool. Questa sezione descrive come queste proprietà si traducono in pattern di deployment concreti: il canale di comunicazione con la pipeline (Sezione 6.2.1), i due profili di esecuzione disponibili (Sezione 6.2.2), e la variabilità di copertura in funzione del contesto organizzativo (Sezione 6.2.3).

6.2.1 Semantic Exit Codes come Canale di Comunicazione con la Pipeline (D6.P1)

Al termine di ogni esecuzione, il tool comunica il proprio esito attraverso quattro codici di uscita con significato preciso: 0 indica che tutti i test attivi hanno prodotto `PASS` o `SKIP`;

1 indica che almeno un test ha rilevato una violazione (**FAIL**); 2 indica che nessun test ha rilevato violazioni ma almeno uno ha prodotto **ERROR**, segnalando un malfunzionamento parziale della pipeline; 10 indica un errore infrastrutturale nelle fasi di avvio che ha impedito l'esecuzione dei test. Quando un assessment presenta sia **FAIL** che **ERROR**, il codice restituito è 1: le violazioni rilevate prevalgono sui malfunzionamenti parziali.

La separazione tra 1 e 10 ha un valore operativo preciso: rilevare una violazione e non avviare l'assessment sono eventi con cause e azioni correttive radicalmente diverse. Il primo richiede di analizzare e correggere il target; il secondo richiede di diagnosticare il deployment della pipeline. Con codici semanticamente distinti questo routing è automatizzabile senza leggere i log.

Sistemi di CI/CD come GitHub Actions o GitLab Continuous Integration (CI)¹ trattano qualsiasi exit code diverso da zero come fallimento del passo: un'esecuzione di **apiguard run** che restituisce exit 1 blocca la pipeline senza configurazione aggiuntiva, e il report rimane disponibile come artefatto per l'analisi post-hoc. Il gate decisionale è il codice di uscita; il report fornisce il dettaglio dei finding per la remediation.

La granularità attuale tratta ogni **FAIL** con lo stesso peso: una violazione P0 su un endpoint di autenticazione e una P3 su una configurazione subottimale producono entrambe exit 1. Rendere il predicato di blocco configurabile per livello di priorità è un'estensione operativa discussa nella Sezione 6.3.1.

6.2.2 Quality Gate a Due Velocità: Fail-Fast e Assessment Completo (D4.P8)

Il tool supporta due profili di esecuzione distinti configurabili sullo stesso file. Il comportamento di default esegue tutti i test attivi indipendentemente dagli esiti intermedi e produce un report completo su tutti i domini: è appropriato per gli assessment periodici su versioni consolidate del software, dove la completezza dell'evidenza ha valore autonomo e ogni dominio deve produrre risultati per supportare le decisioni di correzione.

La seconda modalità, attivata con `execution.fail_fast: true`, risponde a un'esigenza diversa. In un workflow di integrazione continua in cui ogni proposta di modifica viene verificata prima di essere integrata nel ramo principale, attendere il completamento di un assessment da cinque minuti per bloccare su una violazione rilevabile nei primi test P0 è un costo non proporzionale alla decisione. Con `fail_fast` attivo, la prima violazione confermata su un test di priorità P0 interrompe l'esecuzione e il processo termina con exit 1 senza avviare i test rimanenti. La condizione di interruzione si applica anche agli

¹GitHub Actions e GitLab CI sono piattaforme di integrazione continua che eseguono pipeline di build, test e deployment automatizzato al variare del codice sorgente. Entrambe trattano qualsiasi exit code diverso da zero come fallimento del passo, bloccando l'avanzamento della pipeline.

ERROR sui test P0, perché una verifica perimetrale incompleta per malfunzionamento è operativamente equivalente a una violazione: in entrambi i casi il controllo non produce la garanzia attesa. Il wall-clock si riduce da circa cinque minuti a pochi secondi nel caso peggiore.

6.2.3 Variabilità di Privilegio nei Deployment Industriali

Il meccanismo del box gradient descritto nella Sezione 3.3.2 ha una conseguenza operativa diretta nei deployment industriali: la profondità dell'assessment si adatta automaticamente alle precondizioni di accesso disponibili nel contesto organizzativo specifico, senza richiedere versioni diverse del tool o profili di configurazione strutturalmente distinti. La variazione è nella configurazione fornita nel `config.yaml`, che riflette le precondizioni effettivamente disponibili, non una scelta esplicita di modalità. Lo stesso tool che in un ambiente con accesso completo esegue l'intera matrice di domini riduce automaticamente la propria copertura in contesti più vincolati, documentando ogni riduzione tramite **SKIP** con motivazione esplicita: l'analista vede esattamente quali garanzie sono state verificate e quali non erano applicabili nel contesto dato.

6.3 Sviluppi Futuri

Le traiettorie di sviluppo si articolano su due scale temporali con natura distinta. Le estensioni operative seguono direttamente dall'architettura corrente: i punti di estensione sono già predisposti, i contratti tra i componenti sono già definiti, e la mancata implementazione riflette scelte di priorità, non vincoli architetturali. Le traiettorie di ricerca pongono invece problemi aperti che richiedono analisi formale o validazione sperimentale indipendente prima di poter essere affrontati concretamente, e in alcuni casi costituiscono oggetti di studio autonomi.

6.3.1 Estensioni Operative

L'architettura corrente è stata progettata per essere estesa: la tassonomia a otto domini definisce uno spazio di coverage più ampio di quello attualmente coperto, i contratti di **BaseTest** e **BaseConnector** permettono l'aggiunta di test e tool esterni senza modificare il core, e il DAG scheduler è già predisposto per accogliere nuove dipendenze. Le estensioni descritte di seguito completano la copertura della suite e rafforzano l'integrazione con le pipeline industriali; in tutti i casi, i punti di ancoraggio nell'architettura sono predisposti.

Copertura dei test e connectors. Sul fronte della copertura dei domini, il Dominio 5 (Observability) è l'unico senza test attivi nella fase corrente: i test 5.1 (audit logging) e 5.2 (alert real-time) sono già specificati nella metodologia e richiedono l'aggiunta di un log aggregator e un sistema di alerting al testbed, componenti che rimangono architetture isolati dal core. Per i Domini già parzialmente coperti, i test pianificati ma non ancora implementati si inseriscono nel DAG esistente senza interventi sull'ordinamento topologico: i test 1.2 e 1.3 sul ciclo di vita del JWT dichiarano `depends_on = ["1.1"]` e attendono il token prodotto da quel test, esattamente come già accade per i test 1.4 e 2.1 in Phase B. Il Dominio 2 (Authorization) copre attualmente solo il test 2.1; i test 2.2–2.5 con il connector OFFAT per la generazione dinamica di probe BOLA/Insecure Direct Object Reference (IDOR) completerebbero la copertura del controllo accessi a livello di oggetto. I Domini 3, 6 e 7 hanno test pianificati che richiedono rispettivamente *Schemathesis*² per l'injection testing, un connector socket per HTTP request smuggling, e connector Vegeta e Interactsh per race condition e SSRF blind.

Due di questi connector hanno caratteristiche tecniche che vale la pena evidenziare. Il *VegetaConnector* per il test 7.3 (Time-of-Check to Time-of-Use (TOCTOU)) risolve un limite strutturale del runtime Python: le richieste simultanee con sincronizzazione sub-millisecondo richieste da questo test sono inaffidabili con il Global Interpreter Lock (GIL) attivo; delegare la generazione del carico a Vegeta, scritto in Go con goroutine native, ottiene la finestra di concorrenza necessaria. Il connector è riutilizzabile dal test 4.1 per il load testing del rate limiting, in coerenza con il principio di separazione dati/valutazione descritto nella Sezione 4.4.1. L'*InteractshConnector* per il test `ext.7.4` affronta invece un problema di osservabilità: le vulnerabilità SSRF blind non producono output osservabile nel canale normale, e richiedono un oracle fuori banda. Il connector registra un indirizzo controllato su un server interactsh remoto, lo inietta nel probe e verifica se il target ha effettuato una richiesta verso quell'indirizzo; l'evidenza è nel callback ricevuto, non nella risposta al probe.

Integrazione della pipeline. Due estensioni riguardano il canale di comunicazione con la pipeline CI/CD, già descritto nella Sezione 6.2.1. La prima è la granularità degli exit code per livello di priorità, che permetterebbe di distinguere violazioni P0 (bloccanti in qualsiasi scenario) da violazioni P2/P3 (segnalabili come avvertimenti senza bloccare il rilascio). La seconda è l'implementazione degli adapter per i cloud provider principali (Amazon API Gateway, Azure APIM), che consentirebbe la copertura `WHITE_BOX` nei deployment cloud managed discussi nella Sezione 6.1.3.

Gerarchia estesa dell'adapter. La tensione descritta nella Sezione 6.1.2 tra portabilità e profondità delle verifiche vendor-specific ammette una direzione di sviluppo che

²L'uso di *Schemathesis* come connector esterno è distinto dall'uso come strumento autonomo discusso nella Sezione 2.4.3: qui opera come fornitore di dati grezzi, la cui valutazione rimane responsabilità del test.

non richiede modifiche al contratto esistente. `BaseGatewayAdapter` rimarrebbe l'interfaccia comune con i metodi generalizzabili; classi intermedie come `BaseKongAdapter` o `BaseAWSAdapter` esporrebbero metodi vendor-specifici per i test che li richiedono. Un deployment che analizza un gateway Kong istanzierebbe `KongGatewayAdapter`; uno che analizza un gateway AWS istanzierebbe il relativo adapter; entrambi sarebbero selezionabili via configurazione, seguendo lo stesso meccanismo di iniezione dell'adapter attuale. Questa struttura riflette la stessa logica della gerarchia dei connector descritta nella Sezione 4.4.1. Il lavoro necessario riguarda due aspetti: la definizione formale del confine tra metodi generalizzabili, che appartengono a `BaseGatewayAdapter` e sono disponibili a tutti i test, e metodi vendor-specifici, che appartengono al livello intermedio e sono accessibili solo ai test che ne hanno bisogno; e la gestione della compatibilità quando l'architettura evolve, affinché le implementazioni esistenti rimangano valide senza richiedere aggiornamenti in tutto il codebase.

Ottimizzazioni del motore. La struttura a batch del DAG è già parallelizzabile per costruzione, come discusso nella Sezione 4.2.2: i test all'interno di uno stesso batch non hanno dipendenze reciproche. Introdurre un `ThreadPoolExecutor` nella Phase 5 ridurrebbe il wall-clock proporzionalmente al numero di test eseguibili in parallelo; dalla distribuzione della topologia osservata nella Sezione 5.4.2, 16 dei 18 test attivi appartengono alla Phase A e sarebbero candidati alla parallelizzazione. I problemi di thread-safety che questa modifica introduce sulle strutture condivise sono l'oggetto del paragrafo successivo.

6.3.2 Traiettorie di Ricerca Aperta

Le direzioni descritte in questa sezione condividono una caratteristica: l'architettura corrente le abilita o le porta alla luce, ma la loro implementazione richiede analisi formale, scelte di design nuove o validazione sperimentale indipendente. Alcune sono evoluzioni dirette di meccanismi esistenti; altre aprono domande di ricerca autonome.

Parallelizzazione intra-batch e thread-safety. La parallelizzazione intra-batch, identificata come estensione operativa nella Sezione 6.3.1, porta alla luce due problemi di thread-safety che richiedono analisi formale prima dell'implementazione. Il primo problema riguarda l'integrazione dei risultati: ogni test scrive le proprie transazioni su un file `.jsonl` separato nella directory temporanea, ma il merge di questi file nell'evidence store finale e il calcolo del `ResultSet` aggregato avvengono su strutture condivise; con più test concorrenti che completano in ordine non deterministico, la correttezza di queste operazioni di merge richiede coordinazione esplicita. Il secondo problema, più ampio, riguarda la gestione dello stato condiviso nel `TestContext`: token acquisiti, risorse registrate per il teardown e path seed risolti vengono scritti e letti da test distinti; con esecuzione concorrente, accessi non coordinati a queste strutture possono produrre race

condition il cui effetto dipende dall'ordine di schedulazione e non dal comportamento del target. Prima di affrontare la scelta della strategia di coordinazione, andrebbe misurato il guadagno effettivo della parallelizzazione nel contesto specifico: con un'esecuzione dominata dalla latenza di rete verso il target, come osservato nella Sezione 5.4.2, il beneficio dipende dalla distribuzione dei tempi di risposta e dalla possibilità di sovrapporre le attese. Senza questa misurazione, la complessità introdotta dalla gestione della concorrenza potrebbe non essere giustificata.

APIGuard come piattaforma modulare. L'architettura corrente è costruita attorno al paradigma REST/OAS, che costituisce l'unico protocollo per cui il tool produce assessment completi. Classi di API con semantiche diverse, endpoint GraphQL, servizi gRPC, sistemi event-driven basati su AsyncAPI, richiedono meccanismi di scoperta della superficie di attacco, costruzione delle probe e interpretazione delle risposte radicalmente diversi. La traiettoria di ricerca è trasformare APIGuard Assurance da tool monoprodotto a piattaforma di assurance modulare: il core, composto da engine, DAG scheduler, evidence store e report, rimarrebbe invariato, mentre i moduli di protocollo, incluso quello REST/OAS attuale, verrebbero trattati come plugin intercambiabili. In questo modello, un modulo per GraphQL contribuirebbe i propri test e connector rispettando i contratti di **BaseTest** e **BaseConnector**, un operatore potrebbe comporre la suite combinando moduli per i protocolli rilevanti nel proprio contesto, e l'architettura del core rimarrebbe invariata. Le sfide di ricerca principali riguardano due fronti: la specifica formale del contratto tra modulo e core, che deve garantire compatibilità al variare delle versioni e permettere l'aggiunta di moduli senza richiedere modifiche al core; e la definizione di un modello di evidenza sufficientemente astratto da rappresentare semantiche di protocollo diverse, garantendo che i finding prodotti da moduli distinti siano confrontabili nel report finale.

Coverage augmentation progressiva. La specifica OpenAPI definisce la superficie di attacco con cui il tool inizia ogni run, ma durante l'esecuzione il sistema raccoglie osservazioni sul comportamento reale del target che potrebbero estendere quella superficie. I test del Dominio 0 scoprono endpoint attivi non dichiarati nella specifica, i test di autenticazione producono token e schemi di risposta osservabili, i test a visibilità piena leggono la configurazione effettiva del gateway. La traiettoria di ricerca è usare queste osservazioni come input per i test successivi nella stessa run, trasformando la superficie di attacco da un'istantanea statica iniziale a una struttura che si arricchisce progressivamente al procedere dell'esecuzione: un endpoint scoperto nei test iniziali potrebbe alimentare i probe dei test successivi, che attualmente lo ignorano perché non dichiarato nella specifica di partenza. La sfida centrale è metodologica: definire quali osservazioni siano abbastanza affidabili da essere usate come input per test successivi, come propagarle nel DAG garantendo la riproducibilità, e come distinguere endpoint genuinamente non documentati da endpoint inaccessibili per mancanza di permessi.

Capitolo 7

Conclusioni

Tre tensioni hanno orientato le scelte di questo lavoro: l'incapacità degli approcci tradizionali di rilevare vulnerabilità semantiche nelle API REST, l'assenza di strumenti che combinino agnosticismo applicativo con testing guidato dal contratto formale, e la non confrontabilità dei risultati di un assessment tra esecuzioni successive. Il framework sviluppato risponde a ciascuna con meccanismi architetturali verificabili empiricamente. Questo capitolo ripercorre quella risposta, ne valuta la portata effettiva e colloca il contributo nel contesto più ampio della sicurezza come proprietà misurabile del processo di sviluppo.

7.1 Sintesi del Lavoro e Risoluzione del Problema Iniziale

La cecità semantica degli approcci tradizionali, analizzata nel Capitolo 2 come distanza tra ciò che uno scanner osserva e ciò che una vulnerabilità logica richiede di capire, appartiene alla struttura del paradigma: questa barriera persiste indipendentemente dal volume di payload generati, perché un fuzzer che ignora il contratto API affronta il combinatorial wall prima ancora di raggiungere la logica applicativa, con la quota di richieste che violano i tipi e i vincoli enumerati nello schema che viene respinta con HTTP 400 dal layer di validazione. Usare la specifica OpenAPI come oracolo formale, formalizzato nella Sezione 3.1.2, abbatte questa barriera spostando il punto di osservazione dalla sintassi alla semantica: ogni richiesta è costruita nel rispetto dei vincoli contrattuali, ogni risposta è valutata rispetto a ciò che il contratto prevede per quella specifica operazione. Ne diventa raggiungibile la classe di vulnerabilità invisibili al fuzzing cieco: le violazioni di autorizzazione orizzontale (BOLA), l'elusione della revoca dei token e gli abusi di endpoint che accettano URL controllabili dal chiamante, tutte categorie che presuppongono richieste sintatticamente corrette e semanticamente illecite.

L'agnosticismo applicativo, realizzato attraverso la derivazione a runtime di tutta la conoscenza del target dalla specifica OpenAPI e dal `config.yaml` come stabilito nella Sezione 3.1.1, estende questo approccio oltre il singolo caso specifico. Un tool che incorpora conoscenza hardcoded del target, path, strutture dati e comportamenti attesi, è applicabile a quel target e soltanto a quello: sostituita l'applicazione, va riscritto. Un tool che deriva quella conoscenza dalla specifica formale è applicabile a qualsiasi API REST documentata senza modifiche al codice, con il vincolo che una specifica valida sia disponibile; questo vincolo, dichiarato come limite strutturale nella Sezione 6.1, appartiene al paradigma contract-driven nel suo complesso. La profondità dell'analisi si preserva perché il contratto governa la costruzione delle richieste e la valutazione delle risposte con la stessa precisione indipendentemente dal target. L'approccio ibrido risultante occupa lo spazio che i tool vendor-specific e i fuzzer generici lasciano scoperto congiuntamente: i primi sono precisi ma vincolati a un sistema specifico, i secondi sono portabili ma privi di conoscenza contrattuale.

La terza tensione, la non confrontabilità tra esecuzioni successive che esclude il penetration testing tradizionale dall'integrazione come quality gate, trova risposta nella riproducibilità deterministica discussa nella Sezione 3.1.4 e validata empiricamente nella Sezione 5.3. Un framework che produce risultati identici su esecuzioni successive dello stesso target non afferma la riproducibilità come proprietà attesa: la dimostra come invariante misurabile, condizione necessaria perché un assessment automatizzato possa assumere il ruolo di quality gate in una pipeline di sviluppo continuativo.

7.2 Contributi Scientifici e Ingegneristici

I contributi di questo lavoro si articolano su due piani distinti che si rinforzano senza sovrapporsi: la formalizzazione metodologica, che produce strumenti analitici riutilizzabili indipendentemente dall'implementazione specifica, e la realizzazione ingegneristica, che dimostra la concreta praticabilità di quella metodologia con le garanzie di qualità richieste da un artefatto industriale.

Sul piano metodologico, tre contributi si stratificano in ordine crescente di specificità implementativa. La tassonomia a otto domini di assurance, descritta nella Sezione 3.3.1, definisce cosa misurare: organizza la superficie di sicurezza di un'API esposta tramite gateway in categorie semanticamente distinte, ciascuna con le proprie precondizioni di accesso e i propri criteri di valutazione, applicabile a qualsiasi assessment strutturato indipendentemente dagli strumenti impiegati. Il box gradient, formalizzato nella Sezione 3.3.2, aggiunge la dimensione del privilegio e definisce come accedere alla misura: la classificazione Black/Grey/White Box viene derivata dalle precondizioni concrete che ogni garanzia impone al tester, producendo un mapping analitico tra il livello di visibilità disponibile e la classe di controlli raggiungibile. Le trentanove proprietà architettura-

li distribuite su sette domini, raccolte nella Tabella 3.1 e trattate nei Capitoli 3 e 4, definiscono come costruire il sistema che misura: un vocabolario di design che cattura le decisioni architetturali con le loro motivazioni e i costi accettati, riutilizzabile come modello per sistemi analoghi.

Sul piano ingegneristico, il design di un assessment engine deterministico basato su DAG, descritto nella Sezione 3.2.2, costituisce un contributo architetturale autonomo: la struttura a batch con risoluzione topologica garantisce la correttezza semantica dell'ordine di esecuzione indipendentemente dal numero e dalla natura dei test, una proprietà che uno scheduler a priorità statica non raggiunge perché non possiede la rappresentazione delle dipendenze tra test. La codebase che implementa questo design è stata validata empiricamente contro un target reale: su Forgejo 14.0.3 esposto tramite Kong 3.9 DB-less, due run indipendenti hanno prodotto risultati identici per tutti i contatori aggregati (9 PASS / 7 FAIL / 2 SKIP / 0 ERROR / 98 finding, delta wall-clock inferiore allo 0,3%), come verificato nella Sezione 5.3. La qualità del codice è verificabile per analisi statica: 0 errori mypy in strict mode su 90 file sorgente, come documentato nella Sezione 5.2.2; un tool che si propone come strumento di assurance applica alla propria implementazione lo stesso rigore che applica ai target che valuta.

7.3 Considerazioni Finali sull'Assurance Continuo

La differenza tra il penetration testing tradizionale e l'automated security assurance è epistemica, non di scala. Il penetration testing produce un referto: un'osservazione puntuale condotta in un momento specifico, da un operatore con un insieme specifico di conoscenze e tecniche; rieseguire il test in condizioni nominalmente identiche produce output confrontabili solo per convenzione, perché quella variabilità è incorporata nel metodo. L'automated security assurance produce una garanzia: lo stesso assessment, rieseguito con la stessa configurazione sullo stesso target, produce risultati deterministicamente identici, verificabili da chiunque senza ricostruire il processo mentale di chi l'ha condotto. La riproducibilità dimostrata nella Sezione 5.3 non è una proprietà accessoria del sistema: è la condizione che trasforma un'osservazione in un'affermazione verificabile da terzi.

APIGuard Assurance è la dimostrazione che la sicurezza delle API può essere trattata come una proprietà dell'ingegneria del software: verificabile per analisi statica e per esecuzione, tracciabile attraverso un audit trail forense, integrabile nel ciclo di sviluppo tramite exit code semantici senza richiedere intervento manuale a ogni release. Applicare alle API lo stesso rigore che si applica al codice, dalla verifica dei tipi alla regressione automatizzata fino ai quality gate di pipeline, non è un'aspirazione teorica: è ciò che un framework deterministico e config-driven rende operativamente praticabile. Quando l'assurance è continuo, la sicurezza smette di essere un attributo certificato una volta

Capitolo 7 Conclusioni

all'anno e diventa una proprietà del processo stesso, contestata a ogni commit e confermata a ogni deploy.

Bibliografia